

ÚJRAHASZNOSÍTOTT MOBILTELEFONOKON ALAPULÓ BIZTONSÁGI RENDSZER MEGVALÓSÍTÁSA

REALIZACIJA BEZBEDNOSNOG SISTEMA ZASNOVANOG NA RECIKLIRANIM MOBILNIM TELEFONIMA

IMPLEMENTATION OF A SECURITY SYSTEM BASED ON RECYCLED MOBILE PHONES

Hallgató

VASS RICHÁRD
26223071

Mentor

DR. PLETL SZILVESZTER

Szabadka,
2026

Hallgatói nyilatkozat

Kijelentem, hogy e szakdolgozat saját munkám eredménye. A szakdolgozat elkészítésekor felhasznált szakirodalomra és egyéb eszközökre maradéktalanul hivatkoztam. A Szabadkai Műszaki Szakfőiskola a szakdolgozatomban bemutatott eredményeimet, igényei szerint, szabadon és térítésmentesen felhasználhatja, szem előtt tartva az esetleges titkosításra vonatkozó megállapodást, amely ebben az esetben a szakdolgozat részét képezi.

Szabadkán,
kelt

Hallgató aláírása

Absztrakt

A dolgozat egy önálló fejlesztésű, felhőfüggetlen, lokális hálózaton működő biztonsági kamerarendszer, a SwarmCam tervezését és megvalósítását mutatja be, amelynek célja a háztartásokban feleslegessé vált Android okostelefonok IP kameraként történő újrahasznosítása. A telefonokon futó, nyílt forráskódú Android IP Camera alkalmazás HTTPS protokollon szolgáltatja a hardveresen kódolt H.264 videófolyamot és a készülék állapotadatait; ezeket egy Docker-alapú szerverkörnyezetben futó Frigate NVR dolgozza fel valós idejű, mesterséges intelligencia alapú objektumdetekcióval. A rendszer magját egy aszinkron Python (FastAPI) háttér szolgáltatás képezi, amely automatikus hálózati felderítéssel azonosítja az elérhető kamerákat, MQTT protokollon közvetíti a detektált eseményeket, WebSocket kapcsolaton át valós időben tájékoztatja a React alapú, asztali és mobil elrendezést egyaránt támogató kezelőfelületet, az élő videót pedig újrakódolás nélkül, fMP4 szegmensekként játssza át a böngészőnek. A rendszert egy különálló, YOLO-alapú mikroszolgáltatás egészíti ki tűz- és füstérzékeléssel. A dolgozat bemutatja a rendszer architektúráját, a kommunikációs rétegek (REST, WebSocket, SSE, MQTT) tervezési döntéseit, a konténerizált üzemeltetést, valamint a megvalósítás és a valós környezetben végzett, számszerű mérésekkel alátámasztott tesztelés tapasztalatait. Az eredmények igazolják, hogy elavult mobiltelefonokból, dedikált kamerahardver nélkül is kiépíthető egy megbízható, bővíthető és teljes mértékben a felhasználó tulajdonában maradó megfigyelőrendszer.

Kulcsszavak: Android, e-hulladék, FastAPI, konténerizáció, objektumdetekció

Tartalom

Hallgatói nyilatkozat.....	2
Absztrakt.....	3
Tartalom.....	4
Jelmagyarázat.....	6
Köszönetnyilvánítás.....	8
A szakdolgozat témája.....	9
1. Bevezető.....	11
1.1. A probléma leírása.....	11
1.2. A szakdolgozat céljai.....	11
2. Szakirodalmi áttekintés.....	13
2.1. Az otthoni videómegfigyelés technológiai környezete.....	13
2.1.1. A felhőalapú konzumer rendszerek működési modellje.....	13
2.1.2. Az edge, azaz helyben futó NVR alternatívája.....	13
2.2. Kamerahardver: szabványos IP kamerák és fogyasztói kameraalkalmazások.....	15
2.2.1. Az ONVIF szabvány és az RTSP protokoll.....	15
2.2.2. A fogyasztói IP-kamera alkalmazások.....	15
2.3. Objektumdetekció a videóalapú megfigyelésben.....	16
2.3.1. A klasszikus mozgásdetekció és korlátai.....	16
2.3.2. Konvolúciós objektumdetektorok.....	17
2.3.3. A Frigate detekciós pipeline-ja.....	17
2.4. Kommunikációs minták elosztott rendszerekben.....	18
2.5. Konténerizáció és szolgáltatásorientált üzemeltetés.....	19
2.6. Elektronikai hulladék és a mobileszközök újrahasznosítása.....	19
3. Problémamegoldás.....	21
3.1. Követelmények és tervezési kényszerek.....	21
3.2. A SwarmCam architektúrája.....	21
3.3. A hálózati és kommunikációs réteg tervezése.....	23

3.4.	Konténerizáció és üzemeltetés.....	25
3.5.	Implementáció.....	26
3.5.1.	Automatikus hálózati felderítés.....	26
3.5.2.	A kamera-protokoll cseréje és annak következményei.....	28
3.5.3.	Élőkép: az MJPEG-proxytól a go2rtc MSE-ig.....	29
3.5.4.	A kameraforgatás megvalósítása.....	32
3.5.5.	A backend és a valós idejű kommunikáció.....	33
3.5.6.	Tűz- és füstérzékelés.....	35
3.5.7.	A kezelőfelület.....	37
3.5.8.	Mobilnézet.....	41
3.5.9.	Hitelesítés és biztonsági megfontolások.....	44
3.6.	Tesztelés és mérési eredmények.....	45
3.6.1.	A mérési környezet és a módszertan.....	45
3.6.2.	Erőforrás-használat a kameraszám függvényében.....	47
3.6.3.	Detektálási sebesség: GPU-gyorsítás és processzoros futtatás.....	48
3.6.4.	Eseménykéseleltetés és az állapotfigyelés pontossága.....	50
3.6.5.	Az élőkép architektúrájának mérése.....	51
3.6.6.	A képsebesség fényfüggése – a telefonoldali korlát.....	51
3.6.7.	Megfigyelt hibamódok.....	53
3.7.	A megoldás korlátai.....	53
3.8.	Tapasztalatok és tanulságok.....	54
4.	Összegzés.....	56
4.1.	Jövőbeli fejlesztési lehetőségek.....	56
	Irodalom.....	58
5.	Mellékletek.....	60
5.1.	Nyomtatott mellékletek.....	60
5.2.	Elektronikus mellékletek.....	60

Jelmagyarázat

Az alábbi táblázat a szakdolgozatban használt jelöléseket és rövidítéseket, valamint azok feloldását tartalmazza.

Jel/Rövidítés	Értelmezés
AGPL	Affero General Public License (szoftverlicenc)
AI	Artificial Intelligence (mesterséges intelligencia)
API	Application Programming Interface (alkalmazásprogramozási felület)
CPU	Central Processing Unit (központi feldolgozóegység)
DNS	Domain Name System
fMP4	Fragmented MP4 (töredezett MP4 konténerformátum)
FPS	Frames Per Second (képkocka/másodperc)
GPU	Graphics Processing Unit (grafikus feldolgozóegység)
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure (titkosított HTTP)
ICE	Interactive Connectivity Establishment (kapcsolatfelépítési eljárás)
IP	Internet Protocol
JPEG	Joint Photographic Experts Group (képtömörítési szabvány)
JSON	JavaScript Object Notation (adatcsere-formátum)
JWT	JSON Web Token
MIT	Massachusetts Institute of Technology License (megengedő szoftverlicenc)
MJPEG	Motion JPEG
MQTT	Message Queuing Telemetry Transport
MSE	Media Source Extensions (böngészős médialejátszási API)
NAT	Network Address Translation (hálózaticím-fordítás)
NVR	Network Video Recorder (hálózati videórögzítő)
ONNX	Open Neural Network Exchange (modellcsere-formátum)
ONVIF	Open Network Video Interface Forum
PTZ	Pan-Tilt-Zoom (forgatható, dönthető, zoomolható kamera)
REST	Representational State Transfer
RTP	Real-time Transport Protocol
RTSP	Real Time Streaming Protocol

SDP	Session Description Protocol (médialeíró formátum)
SOAP	Simple Object Access Protocol
SPA	Single Page Application (egyoldalas alkalmazás)
SSE	Server-Sent Events
TCP	Transmission Control Protocol
TPU	Tensor Processing Unit
UDP	User Datagram Protocol
URL	Uniform Resource Locator
VPN	Virtual Private Network (virtuális magánhálózat)
YOLO	You Only Look Once (objektumdetekciós model család)

Köszönetnyilvánítás

Köszönettel tartozom mentoromnak, dr. Pletl Szilveszternek a szakdolgozat készítése során nyújtott szakmai iránymutatásért és a tervezetre adott részletes, építő jellegű észrevételekért. Köszönöm továbbá családomnak a türelmet és a támogatást.

A szakdolgozat témája

A háztartásokban rendszeresen felhalmozódnak lecserélt, de hardveresen továbbra is működőképes Android okostelefonok, amelyek kamerája, Wi-Fi rádiója és akkumulátora évekkel túléli azt az időszakot, amíg a készülék a felhasználó elsődleges telefonjaként szolgál. Ezzel párhuzamosan az otthoni biztonsági kamerázás piacát jellemzően felhőalapú megoldások uralják, amelyek egyszerű üzembe helyezést kínálnak, cserébe a felvétel harmadik fél szerverére kerül, jellemzően előfizetési modellben.

A feladat egy felhőfüggetlen, helyi hálózaton működő, mesterséges intelligenciával támogatott biztonsági rendszer (SwarmCam) tervezése és megvalósítása, amely a háztartásban feleslegessé vált Android okostelefonokat – dedikált kamerahardver beszerzése nélkül – automatikusan bevonja a rendszerbe, és a felvételeket helyben, a felhasználó saját hálózataán dolgozza fel valós idejű objektum- és eseménydetekcióval.

Az újrahasznosított okostelefon a rendszerben többcélú szenzoregységként értelmezendő: a jelen munka elsődlegesen a kameraképre (gépi látásra), valamint a telefon állapotadataira (pl. akkumulátor- és töltésállapot) támaszkodik, de az architektúrát úgy kell kialakítani, hogy a készülék további beépített szenzorai (mikrofon, gyorsulásmérő, giroszkóp, fényérzékelő stb.) a későbbiekben további eseményforrásként legyenek bevonhatók.

A várható eredmény nem létező megoldások átirata vagy összehasonlító elemzése, hanem a hallgató által önállóan tervezett és megvalósított, valós hálózati környezetben tesztelt szoftverrendszer.

A szakdolgozat elkészítéséhez a hallgatónak a következő feladatokat kell elvégeznie:

1. Áttekinteni a felhőalapú és a helyben (edge) futó NVR-rendszerek (pl. Frigate) előnyeit és korlátait, valamint az ONVIF-kompatibilis IP kamerák és a fogyasztói IP-kamera-alkalmazások (pl. IP Webcam) közötti különbséget.
2. Megtervezni a rendszer összetevőkre bontott (Android telefon – AI/rögzítő réteg – üzenetközvetítő – alkalmazáslogika – felhasználói felület) architektúráját, és indokolni a komponensek közötti kommunikációs csatornák (REST, WebSocket, MQTT) megválasztását.
3. Megvalósítani a hálózaton elérhető, IP Webcam alkalmazást futtató Android eszközök automatikus, kézi konfiguráció nélküli felderítését (hálózati discovery), és a talált kamerák bekötését a választott NVR-rendszerbe.
4. Kialakítani a felhőfüggetlen, konténerizált (Docker Compose alapú) szerverkörnyezetet, amely a videófeldolgozást, az eseménykezelést és a felhasználói felületet egyetlen, könnyen telepíthető csomagba szervezi.

5. Kiegészíteni a rendszert egy önálló, gépi tanuláson alapuló mikroszolgáltatással olyan események felismerésére (pl. tűz, füst), amelyeket az alapul szolgáló NVR általános célú objektumdetektora natívan nem támogat.
6. Kialakítani a valós idejű, eseményvezérelt kommunikációs réteget (üzenetbróker és WebSocket kapcsolat) a kamerák állapotváltozásainak és a detektált eseményeknek a felhasználói felület felé történő továbbításához.
7. Elkészíteni a rendszer webalapú kezelőfelületét (dashboard), amely a kamerák élő képét, a rögzített eseményeket és felvételeket, valamint a rendszerbeállításokat egységes felületen teszi elérhetővé.
8. Tesztelni a rendszert valódi otthoni hálózati környezetben, több, különböző generációs Android eszközzel, és dokumentálni a mért eredményeket (erőforrás-használat a kamerák számának függvényében, átlagos esemény-késleltetés, GPU-gyorsítás hatása a detektálási sebességre).
9. Összefoglalni a megvalósított rendszer korlátait és továbbfejlesztési lehetőségeit (pl. a telefonok további beépített szenzorainak – mikrofon, gyorsulásmérő, giroszkóp – bevonása újabb eseményforrásként, dedikált ONVIF kamerák integrációja, mobil értesítések, alternatív hardveres gyorsítás).

Tudományos / szakmai terület: Elosztott rendszerek, számítógépes látás és mesterséges intelligencia (objektumdetekció), hálózati protokollok (REST, MQTT, WebSocket), konténerizáció és szolgáltatás-orientált architektúra, webes felhasználói felületek fejlesztése.

Kiinduló irodalom:

- Frigate NVR – Documentation. Elérhető: <https://docs.frigate.video/>
- OASIS, MQTT Version 5.0 – OASIS Standard. Elérhető: <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>
- ONVIF, ONVIF Core Specification. Elérhető: <https://www.onvif.org/specs/>
- I. Goodfellow, Y. Bengio, A. Courville – Deep Learning. MIT Press, 2016.
- Ultralytics – YOLO Docs. Elérhető: <https://docs.ultralytics.com/>
- V. Forti, C. P. Baldé, R. Kuehr, G. Bel – The Global E-waste Monitor 2020. UNU/UNITAR, ITU, ISWA, 2020.
- FastAPI – Documentation. Elérhető: <https://fastapi.tiangolo.com/>
- Docker – Docker Compose Documentation. Elérhető: <https://docs.docker.com/compose/>

1. Bevezető

1.1. A probléma leírása

Napjaink háztartásaiban rendszeresen halmozódnak fel lecserélt, de hardveresen még teljesen működőképes Android okostelefonok: kameramodul, Wi-Fi rádió és akkumulátor is van bennük, a legtöbb mégis fiókban végzi, vagy elektronikai hulladékként (e-waste) kerül megsemmisítésre. Ezzel párhuzamosan az otthoni és kisvállalati biztonsági kamerázás piacát ma jórészt felhőalapú megoldások uralják: ezek egyszerű üzembe helyezést kínálnak, cserébe a videófolyam a gyártó szerverein landol, jellemzően előfizetéses modellben, ami adatvédelmi kockázatot és folyamatos költséget is jelent a felhasználó számára.

A dedikált, lokálisan futtatható NVR (network video recorder) szoftverek – mint a nyílt forráskódú Frigate – megoldják a felhőfüggést és az AI-alapú eseményfelismerést, de a kameraoldalt továbbra is a felhasználónak kell biztosítani, jellemzően ONVIF-kompatibilis IP kamerák megvásárlásával. Nincs olyan, egyszerűen telepíthető megoldás, amely a meglévő, használaton kívüli okostelefonokat automatikusan, kézi konfiguráció nélkül vonná be egy ilyen rendszerbe.

A SwarmCam erre a hiányra reagál: célja, hogy a hálózaton lévő, nyílt forráskódú Android IP Camera alkalmazást futtató Android eszközöket automatikusan felismerje, és egy éretté vált NVR-technológiával (Frigate) összekapcsolva egy teljesen lokális, felhőfüggetlen, AI-alapú megfigyelőhálózatot („swarmot”) hozzon létre.

1.2. A szakdolgozat céljai

A szakdolgozat fő célja a SwarmCam platform tervezése és megvalósítása, amely a fentebb vázolt problémákra ad gyakorlati megoldást. Ennek rész céljai a következők:

1. Meglévő Android okostelefonok automatikus, kézi konfiguráció nélküli integrálása IP kameraként, hálózati felderítés (discovery) segítségével.
2. Felhőfüggetlen, Docker Compose alapú szerverkörnyezet kialakítása, amely a videófeldolgozást, az eseménykezelést és a felhasználói felületet egyetlen, könnyen telepíthető csomagba szervezi.
3. A Frigate NVR beépítése a valós idejű objektum- és mozgásdetekcióhoz, kiegészítve egy önálló, YOLO-alapú mikroszolgáltatással a tűz- és füstérzékeléshez, amelyre a Frigate natívan nem ad megoldást.

4. Valós idejű, eseményvezérelt kommunikációs réteg kialakítása (MQTT a Frigate és a tűzdetektor felől, WebSocket a böngésző felé), amely a kamerák állapotváltozásait és a detektált eseményeket másodperceken belül eljuttatja a felhasználóhoz.
5. Egy modern, React alapú kezelőfelület (dashboard) fejlesztése, amely a kamerák élő képét, a rögzített eseményeket és felvételeket, valamint a rendszer beállításait egységes felületen teszi elérhetővé.

2. Szakirodalmi áttekintés

A SwarmCam tervezési döntéseit három terület jelenlegi állapota határozza meg. Az első az otthoni videómegfigyelés uralkodó modellje, amely a felvételt a gyártó felhőjébe tereli, és ezzel adatvédelmi, üzemeltetési és költségbeli kötöttségeket teremt. A második a videóalapú eseményfelismerés gépi tanulási háttere: az egyfázisú konvolúciós detektorok mára elérték azt a sebesség-pontosság arányt, amely mellett a valós idejű, több kamerát kiszolgáló detekció egyetlen otthoni gépen is elvégezhető. A harmadik az elektronikai hulladék kérdése – az, hogy a lecserélt okostelefonok hardveresen továbbra is alkalmasak kamerafeladatra.

2.1. Az otthoni videómegfigyelés technológiai környezete

2.1.1. A felhőalapú konzumer rendszerek működési modellje

A kereskedelmi forgalomban kapható okosotthon-kamerák túlnyomó többsége azonos architektúrát követ. A kamera a helyi Wi-Fi hálózatra csatlakozik, majd kimenő irányú, tartós kapcsolatot épít a gyártó felhőszolgáltatásához. A videófolyam és a mozgásérzékelés eredménye ezen a kapcsolaton keresztül hagyja el a háztartást; a felhasználó mobilalkalmazása szintén a felhőn keresztül éri el a kamerát, még akkor is, ha fizikailag ugyanabban a szobában tartózkodik. A detekció, a felvételek tárolása és az értesítések kiküldése a szolgáltató infrastruktúráján történik.

A modell előnye az egyszerűség. Az üzembe helyezés néhány perc, és nem igényel hálózati ismereteket – nincs szükség port-továbbításra, statikus IP-címre vagy dinamikus DNS-re, mert a kapcsolatot mindig a kamera kezdeményezi kifelé. A távoli hozzáférés és a frissítések a szolgáltató felelősségi körébe tartoznak, a detekció pedig központi szervereken fut, így a kamerába olcsó processzor kerülhet.

A hátrányok ugyanennek az architektúrának a következményei. A háztartás legérzékenyebb megfigyelési adata – a lakótér folyamatos videóképe – rendszeresen elhagyja a felhasználó ellenőrzési körét, és harmadik fél szerverén kerül feldolgozásra. A funkcionalitás jelentős része előfizetéshez kötött, a működés pedig az internetkapcsolattól és a szolgáltató életképességétől függ: a termék támogatásának beszüntetésével a hardver használhatatlanná válhat. Végül a felhasználó nem tudja ellenőrizni, milyen feldolgozás történik a felvételen, és az meddig marad meg.

2.1.2. Az edge, azaz helyben futó NVR alternatívája

A fenti korlátokra a helyben futó, hálózati videórögzítő (NVR) szoftverek adnak választ. Az NVR a felhasználó saját gépén fut, a kamerákat a helyi hálózaton éri el, és minden feldolgozást, tárolást és értesítést helyben végez; a videófolyam így soha nem hagyja el a háztartás hálózatát.

A nyílt forráskódú Frigate ebbe a kategóriába tartozik, és több szempontból is kiemelkedik a mezőnyből [1]. Első osztályú polgárként kezeli az objektumdetekciót: nem utólag ráépített kiegészítőként, hanem a felvételi logika alapjaként. Először olcsó mozgásmaszkkal szűri a képkockákat, és csak a mozgást tartalmazó régiókat adja át a neurális hálónak, majd a találatokat objektumkövetéssel eseményekké fűzi össze. Az események MQTT-n publikálódnak, ami a belső adatbázis ismerete nélkül teszi lehetővé a külső integrációt. A detektálás processzoron, NVIDIA GPU-n, Google Coral TPU-n vagy Intel OpenVINO-val is futtatható.

Az edge modell ára az üzemeltetési komplexitás: a felhasználónak magának kell gondoskodnia a szerverről, a lemezterületről és a frissítésekről, a kamerák bekötése pedig kézi konfigurációt igényel – minden kamera RTSP-címét, felhasználónevét és jelszavát fel kell venni a konfigurációs fájlba. Éppen ez az a pont, ahol a SwarmCam hozzáad az alapmegoldáshoz: az automatikus hálózati felderítéssel a kamerák kézi konfiguráció nélkül kerülnek be a rendszerbe.

1. táblázat: A felhőalapú és a helyben futó (edge) megfigyelési modell összehasonlítása

Szempont	Felhőalapú konzumer rendszer	Helyben futó (edge) NVR
A videófolyam útja	Elhagyja a háztartást, a gyártó szerverén dolgozódik fel	Nem hagyja el a helyi hálózatot
Költségmodell	Hardver + havi/éves előfizetés	Egyszeri hardverköltség, előfizetés nélkül
Internetfüggés	A működéshez folyamatos internetkapcsolat kell	Internet nélkül is teljes értékű
Adatmegőrzés	A szolgáltató szabályai és csomagja szerint	A felhasználó dönti el, a lemezterület a korlát
Detekció helye	Központi szerver, korlátlan kapacitással	Helyi gép, a rendelkezésre álló CPU/GPU korlátain belül
Üzembe helyezés	Néhány perc, hálózati ismeret nélkül	Szerver, tárhely és kameránkénti konfiguráció szükséges
Bővíthetőség	A gyártó ökoszisztémáján belül	Bármilyen szabványos vagy egyedi forrás integrálható

Az 1. táblázat a két modell alapvető különbségeit foglalja össze. A SwarmCam az edge oszlopot választja, és a felhőalapú modell egyetlen valódi előnyét – a konfiguráció nélküli üzembe helyezést – az automatikus felderítéssel igyekszik átemelni.

2.2. Kamerahardver: szabványos IP kamerák és fogyasztói kameraalkalmazások

2.2.1. Az ONVIF szabvány és az RTSP protokoll

A professzionális biztonságtechnikai piacon a kamerák együttműködését az ONVIF (Open Network Video Interface Forum) specifikációja szabályozza [3]. Az ONVIF SOAP-alapú webszolgáltatásokban írja le a kamera felderítését, képességeinek lekérdezését, a stream-profilok kezelését és a mozgatható (PTZ) kamerák vezérlését. A gyakorlatban a legfontosabb a Profile S: egy Profile S kompatibilis kamera bármelyik Profile S kompatibilis NVR-hez beköthető.

A videóátvitel ONVIF-kameráknál szinte kivétel nélkül RTSP-n (Real Time Streaming Protocol) történik [21]. Az RTSP vezérlőprotokoll: a leíró (SDP) [23] és a lejátszásvezérlő parancsokat szállítja, a médiaadat RTP-csomagokban [22] halad. A hasznos teher többnyire H.264 videó [27], amelynek a jelen munka szempontjából legfontosabb tulajdonsága, hogy a hardveresen kódolt folyam újrakódolás nélkül továbbadható, tárolható és megjeleníthető – erre épül a 3. fejezetben bemutatott élőkép-architektúra.

Az ONVIF-kamera tehát szabványos és megbízható, de kameránként jelentkező beszerzési költséggel jár – ami éppen ellentétes azzal a céllal, hogy a rendszer meglévő, feleslegessé vált eszközökből épüljön fel.

2.2.2. A fogyasztói IP-kamera alkalmazások

A másik út az, hogy a kamerát meglévő okostelefonból állítjuk elő. Az Android platformon több alkalmazás is kiszolgálja a telefon kameráját hálózaton keresztül; működési elvük közös: miniatűr HTTP-szervert futtatnak a telefonon, és a kamera képét egy vagy több végponton teszik elérhetővé.

A legismertebb ilyen alkalmazás a Pavel Khlebovich által fejlesztett IP Webcam [4], amely MJPEG-folyamot, JPEG-pillanatképeket, valamint egy H.264-alapú RTSP-végpontot is kínál, továbbá HTTP-lekérésekkel vezérelhető beállításokkal (felbontás, minőség, éjjellátó mód, zseblámpa) rendelkezik. Az alkalmazás zárt forráskódú, és kizárólag a Google Play áruházból telepíthető.

A választás végül a DigitallyRefined által fejlesztett, nyílt forráskódú (MIT licenc), F-Droidon is elérhető Android IP Camera alkalmazásra esett [12]. Az alkalmazás alapértelmezés szerint a 4444-es porton, HTTPS-en, önálírt tanúsítvánnyal szolgál ki. Végpontjai közül a nyers, Annex-B formátumú H.264 elemi folyam, az MJPEG-folyam, valamint a telefon állapotát és beállításait JSON-ban visszaadó leíró a legfontosabb. A váltás indoklását – és a feláldozott telemetriai adatokat – a 3.5.2. szakasz részletezi.

2. táblázat: Kamerahardver-alternatívák összevetése a SwarmCam követelményei szerint

Szempont	ONVIF IP kamera	Android telefon kameraalkalmazással
Beszerezési költség	Kameránkénti, új hardver	Nulla, a meglévő eszköz újrahasznosul
Szabványosság	ONVIF Profile S, RTSP – jól dokumentált	Alkalmazásfüggő, nem szabványos HTTP API
Videókodek	H.264 / H.265, hardveres enkóder	H.264, a telefon hardveres enkódere
Tápellátás	PoE vagy hálózati adapter, folyamatos	Töltőről; az akkumulátor pufferként is működik
Kiegészítő szenzorok	Nincsenek (esetleg mikrofon)	Mikrofon, gyorsulásmérő, giroszkóp, fényérzékelő, akkumulátor-állapot
Időjárásállóság	IP66/IP67 kültéri kivitel	Nincs, védett elhelyezést igényel
Élettartam-kockázat	Alacsony	Akkumulátor-degradáció, hőterhelés, elavuló Android-verzió
Automatikus felderítés	ONVIF WS-Discovery	Egyedi fingerprinting szükséges

A 2. táblázat szerint a telefonalapú node nem minden szempontból jobb az ONVIF-kameránál: időjárásállóságban és hosszú távú megbízhatóságban egyértelműen elmarad tőle. Két tulajdonsága viszont pontosan a cél szerinti: nulla beszerzési költséggel áll rendelkezésre, és a kamerán túl további szenzorokat is hordoz.

2.3. Objektumdetekció a videóalapú megfigyelésben

2.3.1. A klasszikus mozgásdetekció és korlátai

A hagyományos riasztórendszerek mozgásérzékelése képkockakülönbségen vagy háttérmodellezésen alapul: a legegyszerűbb megközelítés két egymást követő képkocka pixelenkénti különbségét küszöböli, a fejlettebb módszerek – például a Gauss-keverékmodell alkalmazó háttérseparáció [20] – pixelenként statisztikai modellt tartanak fenn a háttérrel.

Ezek az eljárások olcsók és gyorsak, de nem mondanak semmit arról, hogy mi mozog. A gyakorlatban ez elviselhetetlen mennyiségű téves riasztást eredményez: a szélben mozgó ág, az árnyék, az éjszakai rovar a lencse előtt, a fényváltozás és a rezgés mind mozgásként

jelentkezik. A rendszer így használhatatlanná válik, mert a felhasználó néhány nap után figyelmen kívül hagyja az értesítéseket – ezért a gépi látás bevezetése nem kényelmi funkció, hanem a használhatóság feltétele.

2.3.2. Konvolúciós objektumdetektorok

A mély konvolúciós hálók megjelenése alapvetően változtatta meg a helyzetet [5]. A modellcsaládok két csoportra oszthatók: a kétfázisú detektorok (például a Faster R-CNN [17]) először régiójavaslatokat állítanak elő, majd ezeket osztályozzák – pontosak, de számításigényesek –, míg az egyfázisú detektorok, a YOLO (You Only Look Once) család [16] és az SSD [18], a lokalizációt és az osztályozást egyetlen előrehaladó menetben végzik el, a képet rácsra osztva és ráccsellánként előrejelezve a bounding boxokat, az objektumossági pontszámot és az osztályvalószínűségeket.

A megfigyelési alkalmazásban az egyfázisú detektorok jelentik a gyakorlati választást, mert a képkockánkénti késleltetés közvetlenül korlátozza, hány kamerát képes egyetlen gép kiszolgálni. A YOLO-család modern változatai (v8, v11) 320×320 vagy 640×640 pixeles bemeneten dolgoznak, ami azt jelenti, hogy a detekció a rögzítéshez használt teljes felbontástól függetlenül, egy leskálázott képen fut – ez az elválasztás a jelen rendszer élőkép-architektúrájának is a kulcsa.

A gyakorlati telepítést két tényező befolyásolja. Az egyik a modell futtatókörnyezete: a PyTorch-ban betanított modell exportálható ONNX formátumba [13], illetve TensorRT motorra fordítható NVIDIA GPU-n [31]. A másik a kvantálás: a lebegőpontos súlyok 8 bites egészekre alakítása csökkenti a memóriaigényt és a következtetési időt, és feltétele annak, hogy a modell egy Coral típusú célhardveren [32] egyáltalán elférjen.

2.3.3. A Frigate detekciós pipeline-ja

A Frigate ezeket az építőelemeket háromlépcsős pipeline-ba rendezi, és ennek ismerete nélkül a mért erőforrásadatok sem értelmezhetők. Az első lépcső a mozgásmaszk: minden beérkező képkockán olcsó, klasszikus mozgásdetekció fut. A második lépcső csak akkor indul, ha a maszk talált valamit – ekkor a mozgó régiók kivágatai kerülnek a neurális hálózathoz. A harmadik az objektumkövetés, amely a képkockánkénti találatokat időben összefűzi, így egyetlen áthaladó személy egyetlen eseményt eredményez.

Ennek két, a mérések szempontjából fontos következménye van. A detektor terhelése nem a kamerák számával, hanem a képen zajló mozgás mennyiségével arányos: egy mozdulatlan képet mutató kamera gyakorlatilag nem terheli a neurális hálót. A rendszer emellett két független paraméterrel szabályozható – a detekciós képsebességgel és felbontással –, amelyek a rögzített felvétel minőségét nem befolyásolják.

2.4. Kommunikációs minták elosztott rendszerekben

Egy több komponensből álló, valós idejű rendszerben a komponensek közötti kommunikáció megválasztása önálló tervezési feladat. Az alábbi négy minta mindegyikének megvan a maga szerepe, és a rendszer mind a négyet használja – de mindegyiket csak arra, amire való.

A REST [26] a HTTP kérés-válasz modelljére épülő, szinkron erőforrás-hozzáférés. Állapotmentes, jól gyorsítótárazható, minden eszközön és nyelvben triviálisan implementálható, és a hibakezelése a HTTP-státuszkódokra épülhet. Cserébe a szerver nem tud magától üzeni: minden interakciót a kliensnek kell kezdeményeznie. Olyan műveletekhez alkalmas, ahol a kliens tudja, mikor van szüksége az adatra – például egy eseménylista lekérésére vagy egy beállítás mentésére.

A WebSocket [24] a HTTP-kapcsolat protokollváltásával (Upgrade) létrehozott, tartós, kétirányú csatorna. Az első kézfogás után mindkét fél bármikor küldhet üzenetet, keretenként néhány bájt fejléc-többséggel. Pontosan arra való, amire a REST nem: a szerver által kezdeményezett, folyamatos állapotfrissítésre. Költsége, hogy a kapcsolat állapotot hordoz, tehát a szervernek nyilván kell tartania a kapcsolódott klienseket, és kezelnie kell a bontást és az újracsatlakozást.

Az SSE (Server-Sent Events) [25] a WebSocket egyszerűsített, egyirányú rokona: egyetlen, nyitva tartott HTTP-válaszfolyamon keresztül a szerver szöveges eseményeket küld a kliensnek. Mivel közöségi HTTP-válasz, minden fordított proxyn átmegy, és a böngésző beépített EventSource API-jával kezelhető. Olyan feladatokra ideális, ahol a kliens csak fogad – például egy hosszan futó művelet előrehaladásának megjelenítésére.

Az MQTT könnyűsúlyú, publish-subscribe modellű üzenetprotokoll, amelyet eredetileg alacsony sávszélességű telemetriai alkalmazásokra terveztek [2]. A küldő és a fogadó nem ismeri egymást: a küldő egy témába (topic) publikál, a bróker pedig eljuttatja az üzenetet minden feliratkozóhoz. Ez a szétcsatolás a jelen rendszerben konkrét képesség: két, egymást nem ismerő eseményforrás küldhet eseményt ugyanannak a fogyasztónak.

3. táblázat: A kommunikációs minták és jellemzőik

Minta	Írány	Kapcsolat	Tipikus felhasználás
REST / HTTP	Kliens kérdez, szerver válaszol	Rövid életű, állapotmentes	Erőforrás-lekérés és -módosítás
WebSocket	Kétirányú	Tartós, állapotot hordoz	Folyamatos állapotfrissítés, riasztás
SSE	Csak szerver → kliens	Tartós HTTP-válaszfolyam	Hosszan futó művelet előrehaladása
MQTT	Publikáló → bróker → feliratkozók	Tartós, brókeren keresztül	Szétcsatolt, aszinkron eseményközvetítés

A 3. táblázat mintáit a 3.3. szakasz köti hozzá a rendszer konkrét adatfolyamaihoz.

2.5. Konténerizáció és szolgáltatásorientált üzemeltetés

Egy öt szolgáltatásból álló rendszer kézi telepítése – Python-környezet, Node-eszközlánc, MQTT-bróker, NVR, webkiszolgáló, GPU-illesztők – vállalhatatlan terhet jelentene a felhasználónak, és minden gépen más eredményt adna. A konténerizáció erre ad választ: a szolgáltatások a függőségeikkel együtt, egymástól izoláltan, de a gazdagép kerneljét megosztva futnak. A Docker Compose több konténer együttes leírását teszi lehetővé egyetlen deklaratív fájlban [9].

A konténerhálózatok közül kettő releváns. A bridge mód esetén a konténer saját hálózati névtérben, saját IP-címmel fut, és portleképezésen keresztül kapcsolódik a gazdagéphez; ez az alapértelmezett, jól izolált megoldás. A host mód esetén a konténer közvetlenül a gazdagép hálózati névtérét használja: nincs NAT és nincs portleképezés, a konténer pedig ugyanúgy látja a helyi hálózatot, mint a gazdagép – videóalkalmazásban ennek konkrét haszna van.

2.6. Elektronikai hulladék és a mobileszközök újrahasznosítása

A mobiltelefonok gyors modellváltása globálisan is mérhető környezeti terhet jelent. A Global E-waste Monitor 2020 adatai szerint a világ éves elektronikaihulladék-termelése az 50 millió tonnás nagyságrendben mozog és növekszik, miközben a szabályozott újrahasznosítás ennek töredékét fedi le [7]. Az okostelefon ezen belül külön kategóriát alkot: nagy értékű anyagokat tartalmaz, mérete miatt viszont gyakran a fiókban vagy a kommunális hulladékban végzi.

A jelen munka szempontjából nem az anyagvisszanyerés, hanem az újrahasználat a releváns. Egy három-négy éves okostelefon hardvere lényegében sértetlen: a kameramodul felbontása és fényereje meghaladja a belépő szintű biztonsági kamerákét, a Wi-Fi rádió stabil, a rendszerchip pedig hardveres H.264 enkódert tartalmaz. A készülék jellemzően nem meghibásodás miatt kerül lecserélésre, hanem mert az akkumulátora gyengült és a szoftverfrissítések elmaradnak.

Kamerafeladatban ezek a hiányosságok többnyire nem korlátozók. Az akkumulátor-degradáció nem számít, ha a készülék állandó tápellátáson üzemel – sőt, a maradék kapacitás beépített szünetmentes tápként viselkedik, és rövid áramkimaradást is átvészel. Az elavult Android-verzió sem akadály, amennyiben a kamerát kiszolgáló alkalmazás támogatja; a jelen rendszerben használt alkalmazás Android 7.0 vagy újabb rendszert igényel, ami a gyakorlatban a 2016 után gyártott készülékeket jelenti.

A valós korlátok máshol vannak. Az első a hőterhelés: a folyamatos videókódolás melegíti a készüléket, a tartósan magas hőmérsékleten tartott lítiumion-cella degradációja pedig felgyorsul. A második az időjárás: telefon önmagában nem helyezhető ki védtelen kültéri környezetbe. A harmadik a szoftveres megbízhatóság: az Android agresszív energiagazdálkodása leállíthatja a háttérben futó alkalmazást, a kameraszolgáltatás elakadása esetén pedig távoli újraindításra nincs szabványos mód – ezek a korlátok a 3. fejezet hibamódjai között mérhető formában is megjelennek.

Összefoglalva: az áttekintés három követelményt jelöl ki a következő fejezetben bemutatott rendszer számára. A megfigyelési adat nem hagyhatja el a helyi hálózatot; a használhatóság feltétele a gépi látás alapú eseményszűrés; a kamerahardver költsége pedig nem terhelheti a felhasználót – aminek az az ára, hogy a kamerák bekötését, állapotfigyelését és hibatűrését a rendszernek magának kell megoldania.

3. Problémamegoldás

Ez a fejezet a SwarmCam tervezését és megvalósítását mutatja be, a fejlesztés tényleges menetét követve: a követelményekből és az azokból következő tervezési kényszerekből indul, majd a rétegzett architektúra és a kommunikációs csatornák megválasztása következik, ezt követi a konténerizált üzemeltetési környezet és a legfontosabb komponensek megvalósítása, végül a valós hálózaton elvégzett mérések és a vállalt korlátok.

3.1. Követelmények és tervezési kényszerek

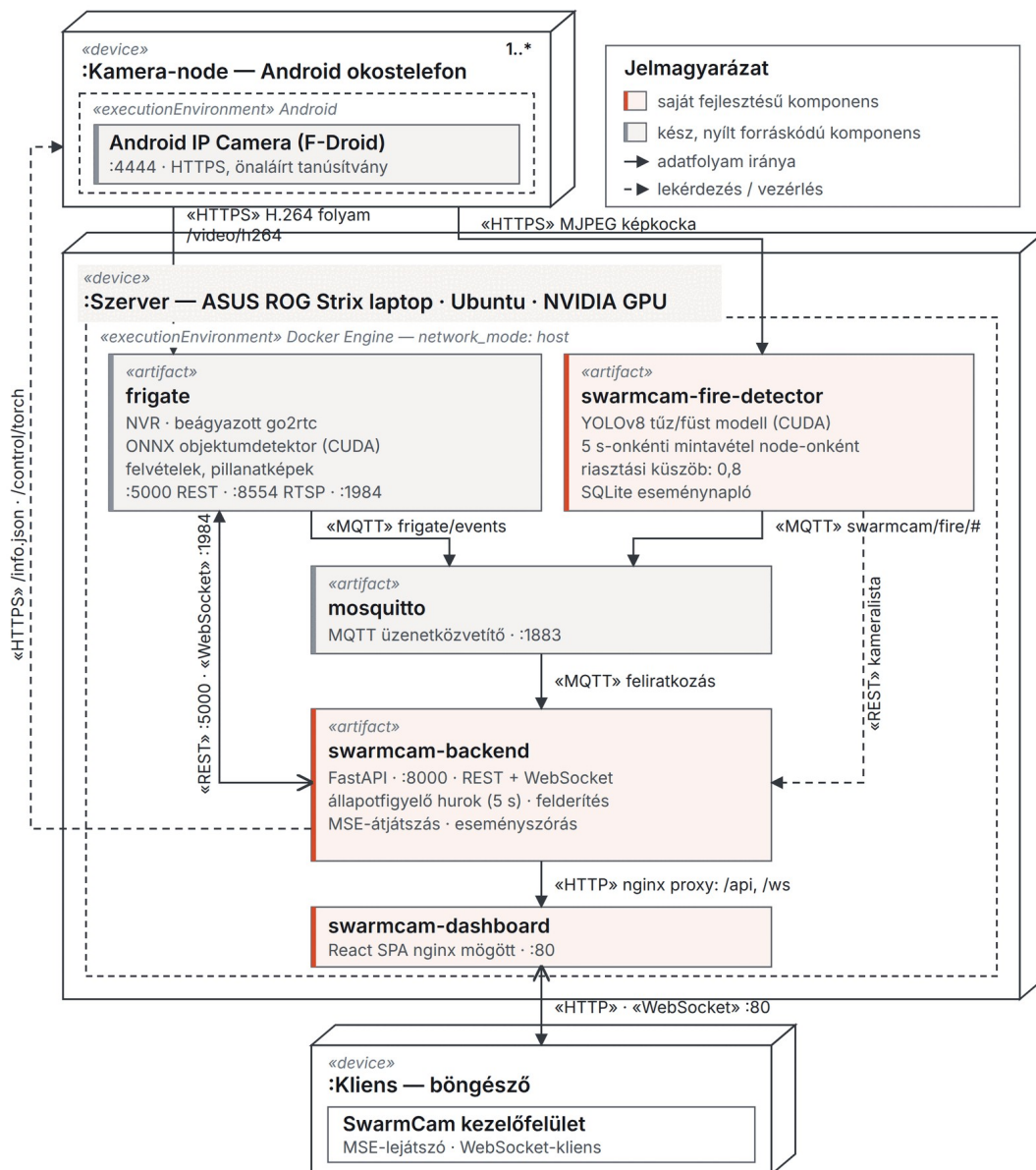
A funkcionális követelmények közvetlenül a feladatkiírásból következnek. A rendszernek kézi konfiguráció nélkül fel kell derítenie a helyi hálózaton elérhető Android kamera-node-okat, valós idejű objektumdetekciót kell futtatnia, és az így keletkező eseményeket – felvétellel és pillanatképpel együtt – böngészhetővé kell tennie. Ki kell egészülnie egy olyan eseményosztály felismerésével, amelyet az alapul szolgáló NVR általános detektora nem támogat; a választott osztály a tűz és a füst. Végül folyamatosan figyelnie kell a node-ok állapotát, és minden változást azonnal meg kell jelenítenie egyetlen webes felületen.

A nem funkcionális követelmények részben ugyaninnen, részben az áttekintő fejezet következtetéseiből erednek. A legfontosabb a felhőfüggetlenség: a rendszer egyetlen komponense sem kommunikálhat külső szolgáltatással, és internetkapcsolat nélkül is teljes értékűen kell működnie. Ezt követi a telepíthetőség – a teljes szerveroldal egyetlen paranccsal álljon fel –, a hibatűrés, hiszen a kamera-node bármikor eltűnhet a hálózatról, valamint az erőforrás-hatékonyság, mert a rendszernek egyetlen, alacsony fogyasztású gépen kell futnia.

Ebből három tervezési kényszer vezethető le. Az első: mivel a kamera-node fogyasztói alkalmazást futtató telefon, nem szabványos ONVIF-eszköz, a felderítés egyedi ujjlenyomat-vizsgálatot igényel. A második: mivel a node bármikor eltűnhet a hálózatról, a rendszer minden komponensének fel kell készülnie a forrás átmeneti hiányára. A harmadik: mivel a detekció számításigénye a limitáló tényező, a videófeldolgozási láncból minden felesleges kódolási és dekódolási lépést ki kell kerülni.

3.2. A SwarmCam architektúrája

A rendszer öt konténerizált szerveroldali szolgáltatásból és a hálózaton szétszórt kamera-node-okból áll. Az architektúra rétegzett: minden réteg csak a közvetlenül alatta és fölötté lévővel kommunikál, jól definiált interfészen keresztül. A telepítési szerkezetet – a fizikai csomópontokat, a rajtuk futó szoftverartefaktumokat és a köztük fennálló kommunikációs útvonalakat – az 1. ábra UML telepítési diagramja foglalja össze.



1. ábra: A SwarmCam telepítési szerkezete UML telepítési diagramon. A csomópontok a fizikai eszközök, a bennük elhelyezett artefaktumok a futó szoftverkomponensek; az élek a kommunikációs útvonalak, a rajtuk feltüntetett sztereotípiával és protokollal. A pirossal jelölt artefaktumok saját fejlesztésűek. Minden komponens a felhasználó saját hálózatán fut; a rendszer semmilyen külső szolgáltatással nem kommunikál.

Az 1. ábra felső csomópontja az adatforrás: az Android IP Camera alkalmazást futtató, újrahasznosított okostelefon. Ebből a rendszer tetszőleges számút kezel – ezt jelöli az 1..* multiplicitás –, és mindegyik ugyanazt a három interfészt kínálja. A nyers, Annex-B formátumú H.264 elemi folyam adja a videót, amelyet a telefon hardveres enkódere állít elő. A JSON leíró a telefon állapotát és beállításait adja vissza, egyben a felderítés ujjlenyomataként is szolgál. A beállítási felület pedig a felbontás, a képsebesség, az aktív szenzor és a zseblámpa távoli állítását teszi lehetővé.

A második réteg az AI és rögzítő réteg, amelyet a Frigate NVR alkot. Fontos részlet, hogy a Frigate nem közvetlenül a telefonhoz csatlakozik: a beágyazott go2rtc komponens [14] veszi át a telefon H.264 folyamát, és helyi RTSP-forrásként szolgálja ki újra. A Frigate ezt a helyi RTSP-forrást dekódolja, futtatja rajta az objektumdetekciót, és menti a klipeket és pillanatképeket. A közbeiktatott go2rtc három konkrét problémát old meg, amelyeket a 3.5.3. szakasz részletez.

A harmadik réteg az üzenetbróker: a Mosquitto MQTT bróker [30]. Két, egymást nem ismerő eseményforrás – a Frigate és a saját fejlesztésű tűzdetektor – közös gyűjtőpontja. A negyedik réteg az alkalmazáslogika, amelyet a FastAPI backend valósít meg [8]: ez felel az állapotfigyelésért, az eseményátjátszásért, az élőkép-relayért, valamint a Frigate REST API-jának proxyzásáért. Az ötödik réteg a felhasználói felület: a React alapú, egyoldalas dashboard alkalmazás [11].

Az architektúrában külön helyet foglal el a tűz- és füstdetekciós mikroszolgáltatás. Ez nem illeszkedik a fenti rétegsorba, mert nem lineárisan kapcsolódik: a kameralistát a backendtől kéri le, a képet közvetlenül a telefonoktól veszi, az eredményt pedig az MQTT brókerre publikálja. A szolgáltatás így az architektúra bármelyik pontjának kiesését túléli, és nem befolyásolja a Frigate detekciós pipeline-ját.

Saját detekciós motort tudatosan nem írtam. A valós idejű, több kamerát kiszolgáló objektumdetekció – mozgásmaszkkal, objektumkövetéssel, felvételkezeléssel és a hardveres gyorsítók támogatásával – önmagában kitöltene egy teljes szakdolgozatot, és gyengébb eredményt adna a rendelkezésre álló nyílt forráskódú megoldásnál. A munka súlypontja ezért a köré épített integrációs réteg: a hálózati felderítés, az állapotfigyelés, a valós idejű eseményközvetítés, az élőkép-architektúra, a tűzdetekciós kiegészítés és a kezelőfelület.

3.3. A hálózati és kommunikációs réteg tervezése

A komponensek közötti adatcsere négy, eltérő célú csatornán zajlik; a 2.4. szakaszban áttekintett minták mindegyike arra a feladatra került, amelyre való.

4. táblázat: A SwarmCam kommunikációs csatornái és megválasztásuk indoklása

Csatorna	Szereplők	Szállított adat	A választás indoka
REST / HTTP	Frontend ↔ backend, backend ↔ Frigate	Kameralista, események, felvételek, felhasználók, konfiguráció	Szinkron, állapotmentes erőforrás- hozzáférés; a kliens tudja, mikor van szüksége az adatra
WebSocket	Backend → frontend	Kamerastátusz, detektált események, riasztások	A szerver kezdemenyezi a frissítést; egyetlen tartós kapcsolat minden eseménytípusra
WebSocket (bináris)	go2rtc → backend → böngésző	fMP4 videószegekmenek (élőkép)	A meglévő nginx- és hitelesítési útvonalon halad; nincs szükség ICE- tárgyalásra és UDP- portra
SSE	Backend → frontend	A hálózati felderítés élő naplója	Egyirányú, hosszan futó folyamat előrehaladása; közönséges HTTP- válasz, minden proxyn átmegy
MQTT	Frigate és fire- detector → bróker → backend	Objektumdetekciós és tűz-/füstesemények	Két, egymást nem ismerő forrás szétcsatolt közvetítése; a bróker átmeneti backend- kiesést is túlél

A 4. táblázat kapcsán három döntés érdemel magyarázatot.

Az első, hogy egyetlen WebSocket kapcsolat szolgálja ki a kamerastátuszt, a detektált eseményeket és a riasztásokat is. Az üzenettípusonként külön végpont tisztább szeparációt adna, de háromszor annyi kapcsolatot, újracsatlakozási logikát és állapotot jelentene a

kliensben, miközben a három üzenettípus együttes forgalma töredéke annak, amit egyetlen kapcsolat elbír; a szétválasztás így egyetlen type mezővel is elvégezhető.

A második, hogy a felderítés naplója SSE-n és nem WebSocketen halad. A felderítés egyirányú folyamat – a kliens elindítja, majd csak fogad –, és néhány tíz másodpercig tart; egy tartós, kétirányú kapcsolat felépítése ehhez fölösleges, az SSE pedig közösleges HTTP-válasz, amely minden proxyn átmegy.

A harmadik, hogy az eseményátvitel MQTT-n történik, nem a Frigate eseményvégpontjának lekérdezésével. Az MQTT push-alapú, tehát az esemény milliszekundumokon belül eljut a backendhez, a lekérdezés viszont a periódus felével növelné az átlagos késleltetést, ráadásul üresjáratú terhelést okozna. A bróker emellett szétcsatolja a két eseményforrást: a Frigate és a tűzdetektor egymásról tudomást sem véve publikál.

Egy technikai részlet, amely a fenti csatornákat a gyakorlatban összeköti: a dashboardot kiszolgáló nginx [34] konfigurációjában kizárólag a /ws/ útvonal továbbítja a WebSocket Upgrade fejléct, az /api/ nem. Ezért minden WebSocket végpont – a státuszcsatorna és az élőkép-relay egyaránt – ezen az előtagon érhető el. Ez nem esztétikai kérdés: az előtag elhagyása esetén a kapcsolat közösleges HTTP-kérésként érkezne a backendhez, és azonnal lezárulna.

3.4. Konténerizáció és üzemeltetés

A szerveroldali infrastruktúra telepítését és izolálását a Docker Compose végzi, egyetlen parancsból induló, öt konténerből álló együttesként (5. táblázat).

5. táblázat: A SwarmCam Docker-konténerei

Konténer	Alap / build	Hálózat	Feladat
frigate	ghcr.io/ blakeblackshear/ frigate:stable- tensorrt	host, NVIDIA runtime	AI-alapú NVR, go2rtc restream, felvételkezelés
mosquitto	eclipse-mosquitto:2	bridge (swarmcam)	MQTT üzenetbróker
swarmcam-backend	saját build (FastAPI)	host	REST/WebSocket API, felderítés, állapotfigyelés
swarmcam- dashboard	saját build (nginx + React)	host	A statikus frontend kiszolgálása, fordított proxy
swarmcam-fire- detector	saját build (YOLO + paho-mqtt)	host, NVIDIA runtime	Tűz- és füstdetekció

A táblázat két sora tervezési döntést rejt. Az egyik a `network_mode: host` használata négy konténernél: a Frigate-nek a kamerák forgalmát a lehető legkisebb késleltetéssel kell feldolgoznia, a backendnek pedig a helyi hálózatot kell pásztáznia a felderítéshez – ez a Docker NAT-olt bridge hálózatán körülményes volna. Az izolációt így a hálózati névtér helyett a konténerhatár adja; a bróker marad bridge módban, mert nincs szüksége a gazdagép hálózatára.

A másik döntés a Docker-socket (`/var/run/docker.sock`) becsatolása a backend konténerbe. Erre azért van szükség, mert a Frigate a konfigurációs fájlját kizárólag induláskor olvassa be: ha a felhasználó a beállítások oldalon módosít valamit, a változás csak a konténer újraindítása után lép életbe. A megoldás ára, hogy a backend a gazdagép Docker-démonjához fér hozzá – ez a helyi hálózat feltételezése mellett vállalható, internet felőli kitétség esetén viszont nem.

A perzisztens adat három helyen él: a Frigate adatbázisa és a felvételek egy Docker-köteten, illetve a gazdagép fájlrendszerére csatolt könyvtárban; a kameraaliasok és a tűzdetektor eseménynaplója a backend melletti állományokban; a rendszer futásidejű állapota – a kameralista és az állapot-cache – pedig kizárólag a memóriában. Ez utóbbi tudatos egyszerűsítés: a kameralista minden induláskor a Frigate konfigurációjából épül újra, tehát egyetlen igazságforrás van, cserébe a rendszer nem tart fenn hosszú távú állapottörténetet.

3.5. Implementáció

3.5.1. Automatikus hálózati felderítés

A felderítés a `discovery/discovery.py` önálló szkriptben valósul meg, amelyet a backend a `/api/discover/stream` végponton külön folyamatként indít el. Az önálló szkript parancssorból is tesztelhető, egy több száz párhuzamos socketet nyitó pásztázás nem terheli a backend event loopját, a szkript pedig a hibakimenetre írja az előrehaladást – ezt a backend Server-Sent Events formájában továbbítja a felületnek, így a pásztázás élőben követhető.

Maga a felderítés kétlépcsős ujjlenyomat-vizsgálat. Az első lépcső gyors TCP-portpásztázás: a szkript a megadott – vagy a gazdagép interfészeiből automatikusan kiolvasott – alhálózat minden címén megkísérel kapcsolatot nyitni a 4444-es porton, 128 párhuzamos szálon, egy másodperces időkorláttal. Ez egy /24-es alhálózat 254 címét néhány másodperc alatt leszűri.

A második lépcső azonosítja, hogy a nyitott port mögött valóban a keresett alkalmazás fut-e. A szkript HTTPS-en lekéri az eszköz `/info.json` leíróját, és az ujjlenyomatot a válasz szerkezetére alapozza: az Android IP Camera válasza mindig tartalmazza a `cameras` és a `settings` kulcsot egyszerre.

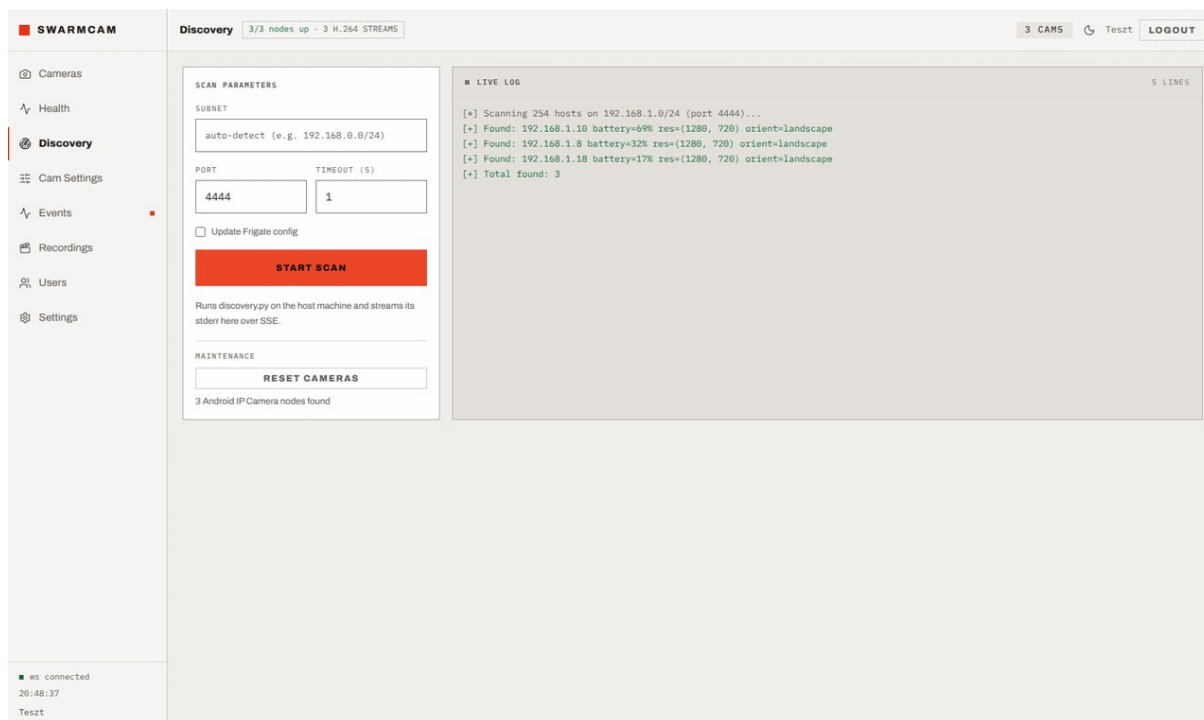
A HTTPS-kérés tanúsítványellenőrzés nélkül (`verify=False`) történik. A tanúsítványt az alkalmazás az első induláskor maga állítja elő, tanúsítóhatóság nem létezik, tehát az ellenőrzésnek nem lenne mit ellenőriznie – a titkosítás kikapcsolása viszont minden telefonon külön kézi lépést igényelne. A `verify=False` a titkosítást nem kapcsolja ki, csak a

serveridentitás hitelesítését hagyja el, a forgalom pedig kizárólag a helyi hálózaton közlekedik.

A leíróból a szkript kiolvassa az akkumulátorszintet, a Wi-Fi jelerősséget, az aktív kameraszenzort és annak felbontását, valamint az orientációt. A felbontás egy tanulságos hibához vezetett. A leíró a folyam méretét auto vagy max kulcsszóként is megadhatja; az első megvalósítás ilyenkor a szenzor legnagyobb támogatott méretét vette tartaléknak – ez azonban a legnagyobb *fényképméret*, nem a videófolyamé. A szám a Frigate detekciós méretébe került, és mivel a Frigate a mozgásdetektálást teljes detekciós felbontáson végzi, egyetlen ilyen kamera – valós, 1280×720-as folyammal – közel háromszáz százalék processzoridőt fogyasztott. A javítás: bizonytalan méret esetén a szkript nem ír detekciós méretet.

A megtalált eszközök determinisztikus nevet kapnak: cam_ előtag után az IP-cím, pontok helyett aláhúzásjelekkel – például cam_192_168_1_8. Így ugyanaz a telefon újraindítás vagy ismételt felderítés után is ugyanazt a belső azonosítót kapja, tehát a hozzá tartozó felvételek és beállítások nem szakadnak el tőle. A névséma emiatt a kameraprotokoll cseréjét is túlélte: a felhasználó által adott, emberi olvasásra szánt nevek migráció nélkül maradtak érvényben.

A felderítés opcionálisan a Frigate konfigurációs fájlját is frissíti: a cameras: szekcióba a belső RTSP-forrásra hivatkozó bejegyzés kerül, a go2rtc.streams: szekcióba pedig a telefon tényleges HTTPS-címe. A két szekció szétválasztása a 3.5.3. szakaszban tárgyalt élőkép-architektúra következménye.



2. ábra: Hálózati felderítés futás közben. A jobb oldali panel a felderítő szkript hibakimenetét mutatja élőben, Server-Sent Events csatornán érkező; a bal oldali panelen a pásztázás paraméterei állíthatók.

A 2. ábra a felderítés felületét mutatja egy befejezett pásztázás után: a napló a pásztázott alhálózatot és a megtalált node-okat jelzi, az azonnal kiolvasott állapotadatokkal együtt.

3.5.2. A kamera-protokoll cseréje és annak következményei

A rendszer eredetileg az IP Webcam alkalmazásra épült, és a fejlesztés későbbi szakaszában került át a nyílt forráskódú Android IP Camera alkalmazásra. A váltás oka nem technikai hiba volt, hanem architekturális következetlenség: egy teljes felhő- és szolgáltatófüggetlenségre törekvő rendszerben idegen elem egy zárt forrású, kizárólag a Google Play áruházból telepíthető komponens. Ez a gyakorlatban is akadályt jelentett: az áruház nem minden készüléken nyitja meg az alkalmazás oldalát, tehát nem minden kéznél lévő telefon volt bevonható.

A csere teljes volt, nem párhuzamos támogatás: az utóbbi megduplázta volna a felderítési és állapotfigyelési kódotakat, miközben a kiváltandó probléma – a régi alkalmazás korlátozott elérhetősége – megmaradt volna. A két protokoll leképezését a 6. táblázat foglalja össze.

6. táblázat: A kamera-protokoll leképezése az alkalmazáscsere során

Funkció	IP Webcam (régi)	Android IP Camera (új)
Alapértelmezett port	8080 (HTTP)	4444 (HTTPS, ön aláírt tanúsítvány)
Ujjlenyomat	/status.json, „curvals” kulcs	/info.json, „cameras” + „settings” kulcs
MJPEG élőkép	/videofeed	/video/mjpeg
Videóforrás a rögzítéshez	Közvetlen RTSP-végpont	/video/h264 nyers folyam, go2rtc újracsomagolással
Zseblámpa	/enabletorch, /disabletorch	?torch=on / ?torch=off
Beállításírás	Kulcsenként külön kérés	Egyetlen kérés több paraméterrel
Licenc és terjesztés	Zárt forrás, csak Play áruház	MIT, F-Droid és forráskód

A csere veszteségekkel is járt. Az új alkalmazás nem szolgáltatja a JPEG-minőség és az éjjellátó mód beállítását, a szabad tárhelyre, az akkumulátor hőmérsékletére és feszültségére vonatkozó adatokat, sem az élő nézők számát; ezek a funkciók kikerültek a rendszerből. A legérzékenyebb veszteség a töltési állapot: a régi protokoll jelezte a töltő csatlakoztatását, így a rendszer figyelmeztetni tudott, ha egy folyamatosan üzemelő telefon tápellátása megszűnt.

Cserébe az új alkalmazás visszaadja a Wi-Fi jelerősséget, támogatja a nagyítást és az expozícióállítást, a videót pedig natív, hardveres H.264 enkóderrel állítja elő. Ez utóbbi a legfontosabb: ez teszi lehetővé, hogy a videó a telefontól a böngészőig egyetlen újrakódolás nélkül jusson el.

3.5.3. Élőkép: az MJPEG-proxytól a go2rtc MSE-ig

Az élőkép megvalósítása a projekt egyik legtöbbet átdolgozott része, és jól illusztrálja, hogyan vezethet egy mérés architektúraváltáshoz. Az első megvalósítás a Frigate hibakeresési MJPEG-folyamát proxyzta a böngészőnek. Ez a megközelítés egyszerű volt és működött, de három szerkezeti problémával járt.

Az első, hogy a Frigate ezen a végponton a *detekciós* képkockákat kódolja JPEG-be. A detekciós felbontás és képsebesség viszont szándékosan alacsony – 640×360 és 5 kép/másodperc –, így az élő nézet lényegesen rosszabb képet mutatott, mint amit a telefon 1280×720-as folyama adott. A második, hogy a JPEG-kódolás nézőnként külön processzorterhelést jelent. A harmadik, hogy az élő nézet minősége és a detekció beállításai összezsugorolódtak.

A megoldás a go2rtc MSE-csatornája. A Frigate imagejébe beágyazva futó go2rtc [14] a telefon H.264 folyamát fMP4 szegmensekké csomagolja át – újrakódolás nélkül –, és WebSocketen szolgáltatja. A backend ezt játssza át a böngészőnek, ahol a Media Source Extensions API-n [15] keresztül a natív H.264 dekóder jeleníti meg. A relay implementációja szándékosan minimális: két, egymással szemben futó továbbító feladat.

Az alábbi részlet ennek a relaynek a lényegi irányát, a go2rtc felől a böngésző felé tartó ágot mutatja. Azért került a dolgozatba, mert rajta látszik, hogy az élőkép-útvonalon a backend semmilyen videóműveletet nem végez: a kód a beérkező kereteket egyetlen ponton sem nyitja fel.

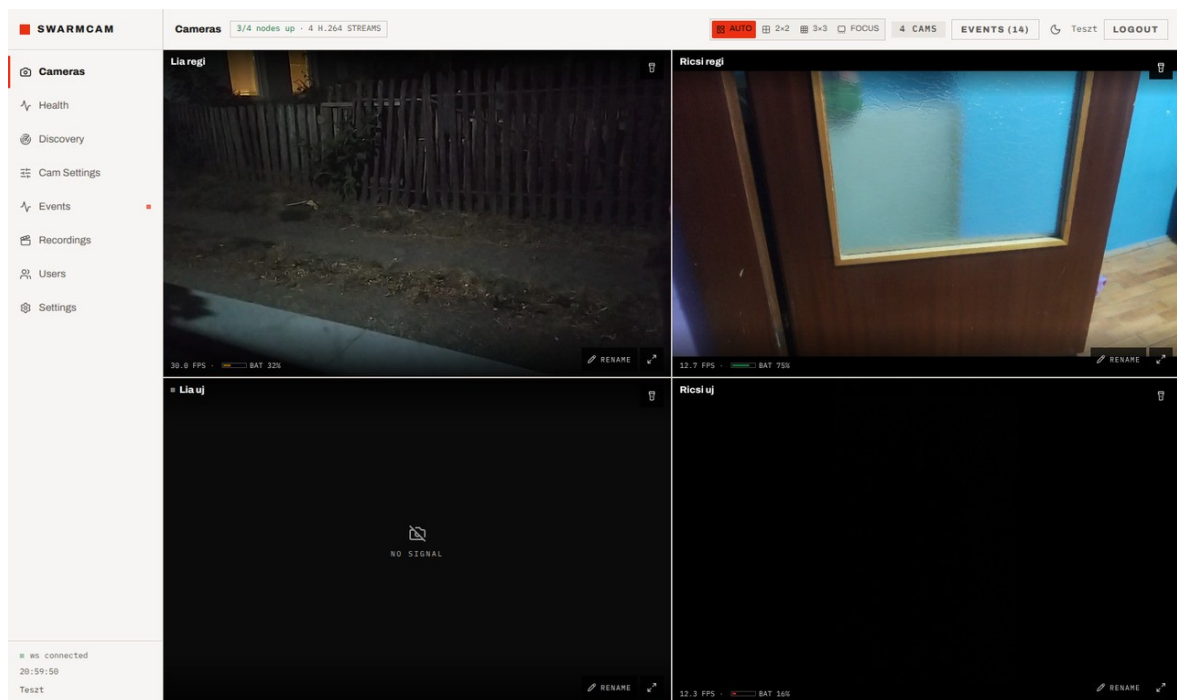
```
@router.websocket("/ws/cameras/{name}/mse")
async def camera_mse(ws: WebSocket, name: str):
    await ws.accept()
    async with websockets.connect(f"{GO2RTC_WS_URL}?src={quote(name)}",
                                max_size=None) as up:
        async def go2rtc_to_client() -> None:
            async for msg in up:
                if isinstance(msg, bytes):
                    await ws.send_bytes(msg)
                else:
                    await ws.send_text(msg)
```

A részleten két dolog a lényeges. Az egyik a send_bytes és a send_text szétválasztása: a go2rtc ugyanazon a kapcsolaton küldi a bináris fMP4 szegmenseket és a szöveges, JSON formátumú vezérlőüzeneteket – a kodekleíró és a lejátszásvezérlést –, a relay pedig mindkettőt a kerettípus megőrzésével adja tovább. Ha a bináris szegmens szöveges keretként érkezne meg, a böngésző MediaSource puffere azonnal hibára futna. A másik a max_size=None: a websockets könyvtár alapértelmezett, egy megabájtos keretmérete a kulcsképkockát tartalmazó, nagyobb szegmenseket eldobná, és a lejátszás néhány másodpercenként megszakadna. A kódban nincs dekódolás, átkódolás és pufferezés – ez az oka annak, hogy az élőkép nézőnkénti költsége a korábbi MJPEG-útvonalhoz képest nagyságrenddel kisebb lett.

A választás mellett a mérés szól: a váltás után az élő nézet a telefon natív felbontásán fut, a végponttól végpontig mért késleltetés 0,03–0,07 másodperc, a Frigate processzorterhelése pedig nézőnként a korábbi huszonhat helyett körülbelül két százalékponttal nő. A számszerű összehasonlítást a 3.6.5. szakasz közli.

A go2rtc a WebRTC-t is támogatja, a választás mégsem az lett. A WebRTC ICE-tárgyalást és UDP-porthasználatot igényel, ami a fordított proxy és a hitelesítési útvonal konfigurációját is érinti. Az MSE ezzel szemben ugyanazon a WebSocketen halad, amely már átmegy a meglévő nginx- és hitelesítési láncon, ugyanúgy újrakódolás nélkül továbbítja a H.264-et, és a helyi hálózaton nem mérhető késleltetéstöbbletet okoz. A WebRTC előnyei – NAT-átjárás, adaptív bitráta – itt értelmüket veszítik.

A régi MJPEG-proxy tartalékútvonalként megmaradt: három sikertelen MSE-csatlakozási kísérlet után automatikusan átveszi a helyét, és a fókusz nézet előnézeti képeit is ez szolgálja ki. Ha a go2rtc-hez vezető út bármely okból nem járható, a felhasználó akkor is lát képet.

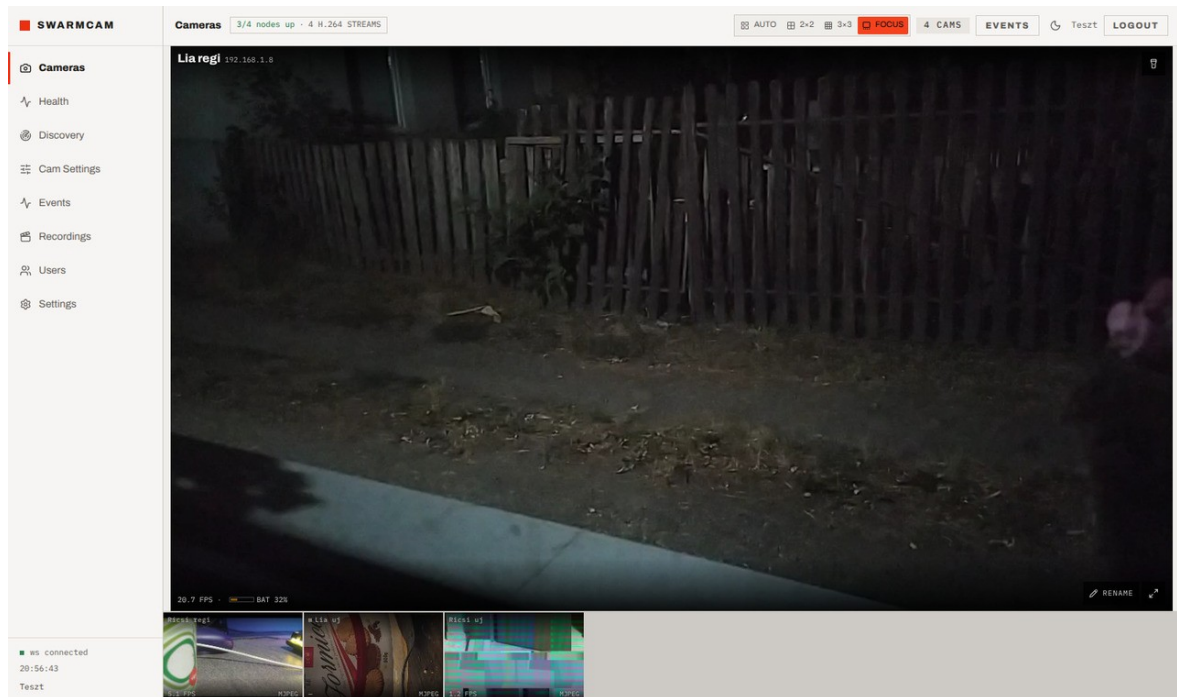


3. ábra: A dashboard Cameras nézete négy regisztrált node-dal. A fejlécben a „3/4 nodes up” jelző mutatja, hogy az egyik telefon éppen nem elérhető; a hozzá tartozó kártya NO SIGNAL állapotot jelez, miközben a többi node élőképe, mért képsebessége és akkumulátorszintje zavartalanul frissül.

A 3. ábra a normál üzemi nézetet mutatja abban a – valós üzemben rendszeresen előforduló – helyzetben, amikor az egyik node nem érhető el. Ez jól szemlélteti a hibatűrési követelmény teljesülését: egyetlen node kiesése nem befolyásolja a többi, a felület pedig egyértelműen jelzi a kiesést.

A kamerarács négyféle elrendezésben használható: automatikus, kétszer kettes, háromszor hármas és fókusz. Az utolsó (4. ábra) egyetlen kamerát nagyít fel, a többi az alsó sávban, előnézeti képként jeleníti meg – ez a folyamatos megfigyelés tipikus munkamódja. Az

előnézeti képeket az MJPEG-útvonal szolgálja ki, mert egyszerre több MSE-folyam futtatása fölöslegesen terhelné a böngészőt.



4. ábra: Fókusz elrendezés. A kiválasztott kamera a teljes területet elfoglalja, a többi node az alsó sávban, előnézeti képként marad követhető; a kiemelt kamera alatt a valós, mért képsebesség és az akkumulátorszint látható.

Egyetlen kamera teljes képernyős megjelenítése külön nézetben is elérhető (5. ábra). Ez a nézet a háttérben futó összes többi élő folyamatot szünetelteti – amit a fejléc külön jelez is –, így a megjelenített kamera a rendelkezésre álló sávszélességet és dekódolási kapacitást egyedül használja.



5. ábra: Teljes képernyős kameranézet. A kép rögzített 16:9 arányban jelenik meg, így az eredeti képarány torzítás nélkül marad; a fejléc jelzi, hogy a háttérben futó élő folyamatok szünetelnek, a lábléc pedig a képsebességet, az akkumulátorszintet és a Wi-Fi jelerősséget mutatja.

3.5.4. A kameraforgatás megvalósítása

A telefonok elhelyezése ritkán teszi lehetővé, hogy a kép eleve a kívánt tájolásban érkezzen. A telefon saját forgatási beállítása nem megoldás: a H.264 folyamra nincs hatással, az alkalmazás csupán lekicsinyíti a képet és fekete sávok közé illeszti – az eredmény felbontásvesztés, forgatás nélkül.

A forgatás ezért a go2rtc feldolgozási láncába került. Forgatás nélkül a go2rtc a telefon folyamatát változatlanul veszi át; forgatás esetén a lánc két tagúvá válik, egy nyers folyammal és egy abból képzett, elforgatott változattal. A forgatás így egyetlen helyen történik, és minden fogyasztó – detekció, felvétel, pillanatkép, élőkép – ugyanazt a képet kapja.

A forgatás átkódolással jár, ez pedig processzoridőbe kerül. A terhelés az NVIDIA videómotorra terelhető: a lánc a képet a GPU dekóderével bontja ki és a h264_nvenc kódolóval állítja elő újra, a képforgatást viszont a processzor végzi, mert a rendelkezésre álló ffmpeg-fordítás nem tartalmazza a forgatás GPU-s megfelelőjét. A mérés szerint egy 720p25-ös folyamnál a szoftveres kódolás egy processzormag 44 százalékát, a GPU-s 6 százalékát igényli, miközben a kimenet képpontra azonos (44,7 dB PSNR, 0,993 SSIM). Három forgatott kameránál ez a Frigate konténer teljes processzorhasználatában körülbelül 590 százalékról 130 százalékra esik vissza.

Az átkódolásnak egy nem kézenfekvő mellékhatása is van: az újracsomagolt folyam elveszíti az eredeti időbélyegeket. A rögzítés ezt nem érzi meg, mert a felvételi ág a képkockákat másolja; a detekciós ág viszont képsebesség-szűrőt alkalmaz, amely az időbélyegek alapján

válogat, és azok híján egyetlen képkockát sem enged tovább. A hiba kifelé alig látszik – a felvétel készül, a lemez telik –, miközben a kamera detekciós képsebessége nulla marad. A megoldás az volt, hogy a forgatott kamerák bemenete olyan ffmpeg-presetet kap, amely a hiányzó időbélyegeket a beérkezés idejéből pótolja. A bemeneti argumentumnak ilyenkor pontosan a preset nevével kell egyeznie: a Frigate csak teljes névegyezés esetén helyettesíti be, egyébként a nevet fájlnévként adja tovább az ffmpeg-nek, és minden kamera újraindulási hurokba kerül.

A forgatás állapotát nem külön fájl, hanem maga a Frigate konfigurációjának forrásleírója tárolja, így nincs két, egymástól elcsúszható igazságforrás.

Fontos korlát, hogy a forgatás nem javítja ki az álló tájolású telefont. Ha a készülék képernyőforgatása állóra van rögzítve, az alkalmazás álló képet illeszt a fekvő képkockába – a mérés szerint a képkocka 68 százaléka fekete sáv –, és ezt a go2rtc-oldali forgatás nem tudja visszanyerni, mert a képpontok már nem léteznek. A megoldás a telefon oldalán van: a képernyőforgatás feloldásával az alkalmazás fekvő képet ad.

3.5.5. A backend és a valós idejű kommunikáció

A backend két háttérfolyamatot futtat, amelyek a rendszer valós idejű viselkedéséért felelnek; mindkettő a FastAPI indulásakor jön létre.

Az első az állapotfigyelő hurok: ötmásodpercenként végigmegy a nyilvántartott kamerákon, mindegyiktől lekéri a /info.json leíró két másodperces időkorláttal, az eredményt a memóriabeli állapot-cache-be írja, és a teljes frissítést egyetlen üzenetben elküldi minden kapcsolódott WebSocket-kliensnek.

Az alábbi részlet ezt a hurkot mutatja. Két sorára érdemes figyelni: a for ciklusra, amely a node-okat sorosan, egymás után kérdezi le, és az utolsó sorra, ahol a HEALTH_INTERVAL szerinti várakozás nem a kör elején, hanem a teljes kör lefutása után következik.

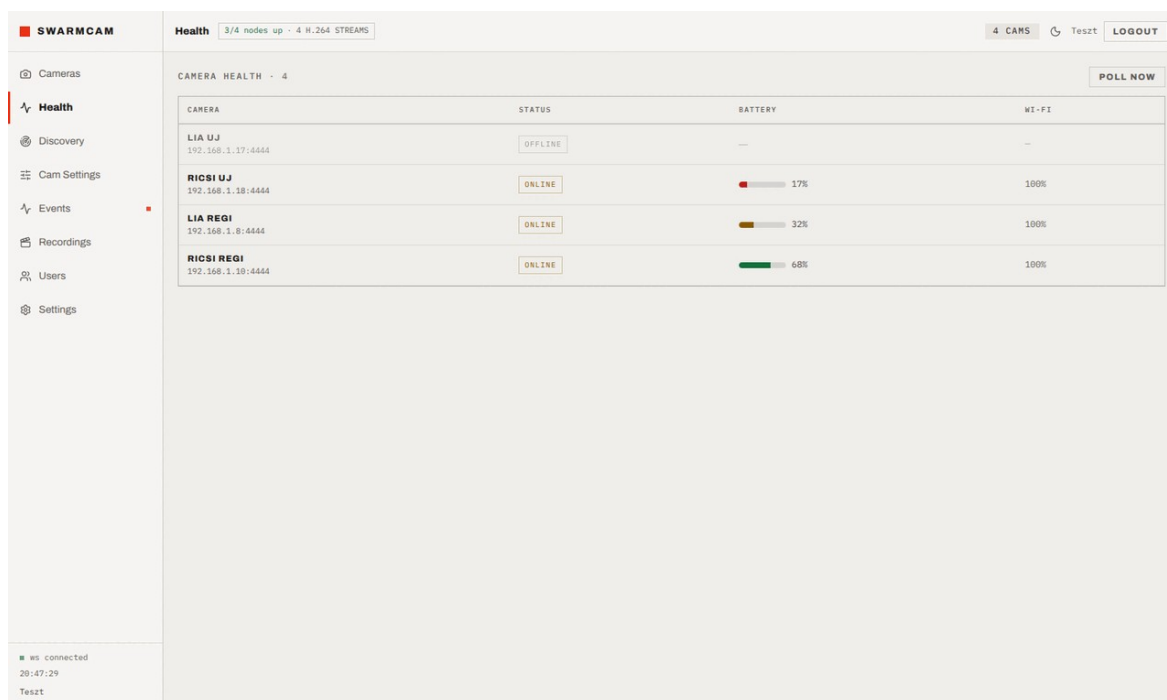
```
async def health_loop() -> None:
    while True:
        updates = []
        for cam in state._last_cameras:
            status = await poll_status(cam["ip"], cam["port"])
            entry = {"ip": cam["ip"], "name": cam["name"],
                    "online": status is not None}
            if status:
                entry.update(_parse_info(status))
            state._health_cache[cam["ip"]] = entry
            updates.append(entry)
        if state._ws_clients and updates:
            msg = json.dumps({"type": "status", "cameras": updates})
            await broadcast(msg)
        await asyncio.sleep(HEALTH_INTERVAL)
```

Ez a két szerkezeti vonás együtt magyarázza a hurok mért viselkedését. Mivel a ciklus minden node választ – a nem válaszolóknál a poll_status kétmásodperces időkorlátjának lejártát – megvárja, és az asyncio.sleep csak ezután fut le, a tényleges periódus nem öt másodperc, hanem a névleges érték és a lekérdezési kör hosszának összege. A 3.6.4. szakaszban mért

7,84 másodperces periódus közvetlenül ebből következik. A `state._health_cache` írása a küldéstől függetlenül, minden körben megtörténik: erre azért van szükség, hogy egy később csatlakozó kliens azonnal teljes állapotképet kapjon, és ne kelljen a következő körre várnia. A szórás pedig ott áll elő, hogy a kör hossza a nem válaszoló node-ok számától függ.

Az öt másodperces periódus kompromisszum. Rövidebb periódus gyorsabb hibafelismerést adna, de feleslegesen terhelné a telefonokat – minden lekérés ébren tartja a rádiót és fogyasztja az akkumulátort –, hosszabb periódus mellett viszont egy kiesett node felismerése észrevehetően lassulna. A tényleges felismerési késleltetés a lekérdezési kör hosszának és az öt másodperces várakozásnak az összege; a mért periódust a 3.6.4. szakasz közli.

Az állapotadatok kiolvasása nem triviális leképezés: az aktív kameraszensor azonosítója a leíróban hol tájolási kulcsszó, hol a szenzorlista egy elemének azonosítója – a kód mindkét esetet kezeli. A tájolás pedig a felhasználó által beállított forgatásból származik, nem a szenzor fizikai beépítési szögéből.



The screenshot shows the 'Health' section of the SWARMCAM interface. It displays a table with 4 columns: CAMERA, STATUS, BATTERY, and WI-FI. The table lists four cameras: LIA UJ (OFFLINE), RICSU UJ (ONLINE), LIA REGI (ONLINE), and RICSU REGI (ONLINE). The battery levels are 17%, 32%, and 68% respectively. The Wi-Fi signal is 100% for all online cameras. A 'POLL NOW' button is visible in the top right corner of the table area.

CAMERA	STATUS	BATTERY	WI-FI
LIA UJ 192.168.1.17:4444	OFFLINE	—	—
RICSU UJ 192.168.1.18:4444	ONLINE	17%	100%
LIA REGI 192.168.1.8:4444	ONLINE	32%	100%
RICSU REGI 192.168.1.19:4444	ONLINE	68%	100%

6. ábra: A Health nézet. Az állapotfigyelő hurok által gyűjtött adatok node-onként: elérhetőség, akkumulátorszint és Wi-Fi jelerősség; a lekérésre nem válaszoló node OFFLINE állapottal, adatok nélkül jelenik meg. A táblázat a WebSocket-en érkező frissítésekkel valós időben változik.

A 6. ábra a rendszer állapotfigyelő nézetét mutatja. A táblázat nem lekérdezésre frissül: az adatok az állapotfigyelő hurok WebSocket-üzeneteiből érkeznek, tehát a felület a böngésző beavatkozása nélkül követi a valóságot.

A második háttérfolyamat az MQTT-hurok: egyetlen tartós kapcsolatot tart fenn a brókerrel, és két témára iratkozik fel, a Frigate objektumdetekciós eseményeire és a tűzdetektor találataira. A Frigate eseményét átalakítja a felület által várt alakra és továbbítja; a

tűzeseményt viszont előbb SQLite adatbázisba menti, az annotált képkockát a fájlrendszerre írja, és csak ezután küldi tovább – felugró riasztásként és eseménylista-bejegyzésként.

A tűzesemény önálló SQLite [33] adatbázisba kerül, nem a Frigate tárolójába. A Frigate adatbázisa belső implementációs részlet: sémáját a fejlesztők verzióról verzióra módosíthatják, és külső események beszúrására nincs dokumentált mód – egy közvetlen írás tehát nem publikus szerződéstől tenné függővé a rendszert, ráadásul írási versenyhelyzetet teremtene. A saját tábla ezzel szemben teljes ellenőrzést ad a séma felett, és feltétele annak, hogy a tűzdetektor a Frigate leállása mellett is működhessen.

3.5.6. Tűz- és füstérzékelés

A feladatkiírás előírja a rendszer kiegészítését egy olyan eseményosztály felismerésével, amelyet az alapul szolgáló NVR általános detektora nem támogat. A választott osztály a tűz és a füst: felismerésük biztonsági rendszerben nyilvánvaló értékkel bír, a Frigate alapmodelljének COCO-címkekészletében [19] viszont nem szerepel.

A megoldás önálló mikroszolgáltatás, nem a Frigate modelljének cseréje: a Frigate egyszerre egy modellt futtat, tehát a tűzfelismerő betöltése az általános objektumdetekció elvesztésével járna. A két feladat két modellt kíván, párhuzamos futtatásukra pedig a külön folyamat a legegyszerűbb mód.

A szolgáltatás működése egyszerű ciklus: a backendtől lekéri az aktuális kameralistát, majd minden kameráról mintát vesz. A mintavétel a telefon MJPEG-folyamából történik – a szolgáltatás megnyitja a folyamatot, az első teljes JPEG-képkockát a formátum kezdő- és zárójelzői alapján olvassa ki, majd azonnal bont –, mert az alkalmazás nem kínál külön pillanatkép-végpontot.

A kivágott képkockán YOLO-modell fut. A modell egy nyilvánosan elérhető, tűz és füst osztályokra betanított súlykészlet [10], amely három osztályt ismer: tűz, füst és egyéb. A szolgáltatás csak az első kettőt veszi figyelembe, és csak akkor publikál, ha a modell konfidenciája eléri a nyolcvan százalékos küszöböt.

Az inferencia a szerver grafikus processzorán fut, ugyanazon az eszközön, amelyet a Frigate detektora is használ. A két modell érdemben nem versenyez: a tűzdetektor kameránként öt másodpercenként egyetlen képkockát dolgoz fel, tehát a GPU-t rövid, ritka lökésekben terheli. Ára a konténer mérete, amely a CUDA runtime miatt mintegy két gigabájtról hatra nő.

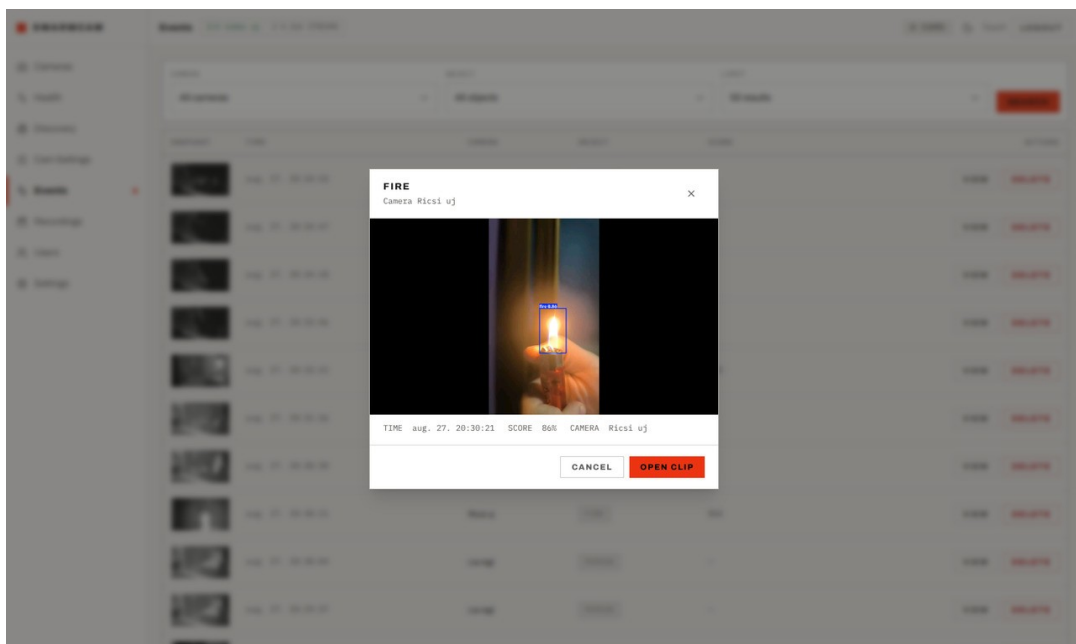
A nyolcvan százalékos küszöb mérési tapasztalat eredménye, nem alapértelmezés: a kezdeti, ötven százalékos küszöb mellett a rendszer éjszaka gyakorlatilag használhatatlan volt. A küszöb hangolását és az azt megalapozó adatokat a 3.6.7. szakasz tárgyalja.

Az alábbi részlet a szolgáltatás döntési pontját mutatja: azt a néhány sort, amely az inferencia nyers kimenetéből eldönti, mi minősül továbbadandó eseménynek.

```
results = model(img, verbose=False)
for r in results:
    for box in r.bboxes:
        conf = box.conf.item()
        if conf >= CONFIDENCE:
            label = model.names[int(box.cls.item())]
            if label.lower() in ["fire", "smoke"]:
                payload = {"camera": cam_name, "label": label.lower(),
                           "score": round(conf, 3)}
                mqtt_c.publish(f"swarmcam/fire/{cam_name}",
                               json.dumps(payload))
```

A szűrés két lépcsős, és a lépcsők sorrendje nem közömbös. Előbb a konfidencia rostál (`conf >= CONFIDENCE`), és csak az ezen átjutó találatoknál vizsgálja a kód a címkét; ez a `CONFIDENCE` konstans az a paraméter, amelyet a 3.6.7. szakasz mérése alapján emeltem ötven százalékról nyolcvanra. A címkeszűrésre azért van szükség, mert a modell harmadik, „egyéb” osztálya a rendszer szempontjából nem esemény – enélkül gyakorlatilag minden feldolgozott képkocka riasztást generálna. A publikálás maga egyetlen MQTT-üzenet a kameránként külön témára: a szolgáltatás nem hívja a backendet, nem ír a közös adatbázisba és a felület létezéséről sem tud. A bróker az egyetlen csatlósi pontja a rendszer többi része felé, és ez teszi lehetővé, hogy a Frigate leállása mellett is működjön.

A találat mellé a szolgáltatás annotált – bounding boxokkal és konfidenciaértékkel ellátott – képkockát is generál, legfeljebb 1280 pixel szélességre kicsinyítve, base64 formában az üzenet részeként. Így a vizuális bizonyíték már a felület első megjelenítésekor rendelkezésre áll, külön lekérés nélkül.



7. ábra: Valós tűzdetekció eseményrészletei. A tűzdetekciós szolgáltatás 86 százalékos konfidenciával azonosította a lángot; a képen a modell által rajzolt bounding box és a hozzá tartozó pontszám látható.

A 7. ábra egy valós, éles környezetben rögzített tűzdetekciót mutat. A találat annotált képkockája a modell által rajzolt bounding boxot és a pontszámot is tartalmazza, így az esemény utólagos ellenőrzése nem igényel külön eszközt.

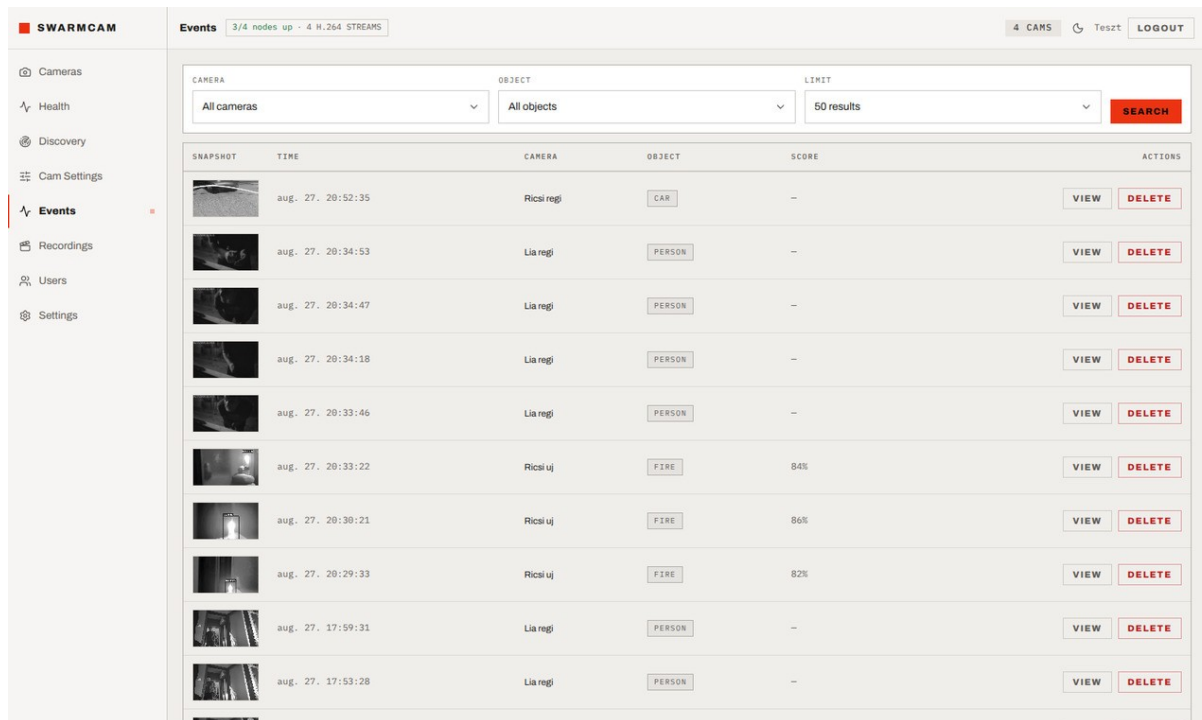
Licencjogi szempontból lényeges, hogy az inferenciát végző Ultralytics keretrendszer [6] AGPL-3.0 licenc alatt áll. Oktatási célú, nem terjesztett rendszernél ez nem akadály; kereskedelmi felhasználás licencvásárlást vagy a keretrendszer cseréjét igényelné – például a súlyok ONNX formátumba exportálásával, ami magát a modellt nem érinti.

3.5.7. A kezelőfelület

A kezelőfelület React és TypeScript alapú, Vite eszközlánccal épített egyoldalas alkalmazás; a megjelenítést Tailwind CSS és shadcn/ui komponensek adják. Útvonalválasztó könyvtárat nem használ – az aktív nézetet egyetlen állapotváltozó tárolja –, mert a dashboard helyi adminisztrációs felület, amelyhez nem tartozik megosztható hivatkozás.

A valós idejű frissítést egyetlen, központi kapcsolatkezelő biztosítja: ez tartja fenn a WebSocket-kapcsolatot, és a beérkező üzeneteket típus szerint osztja szét – az állapotüzenetek a kamerakártyákat és a Health nézetet frissítik, az eseményüzenetek az eseményfolyamba kerülnek, a riasztások felugró értesítésként jelennek meg. A kapcsolat bontását a hook automatikusan kezeli: öt másodperc múlva újracsatlakozik.

Az események két helyen érhetők el: az élő eseményfolyam a kamerarács mellé nyitható oldalpanelen, a történeti böngésző pedig a Frigate eseményadatbázisát kérdezi le a backend proxyján keresztül, kamerára, objektumtípusra és darabszámba szűrhetően.



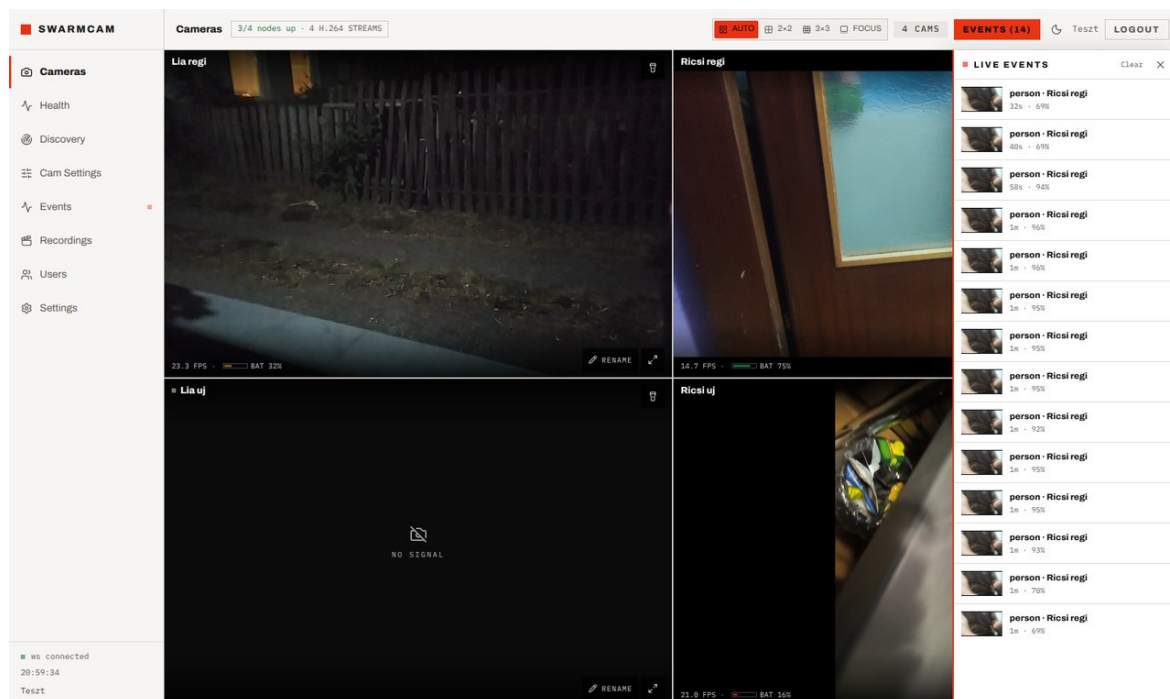
CAMERA	OBJECT	LIMIT	
All cameras	All objects	50 results	SEARCH

SNAPSHOT	TIME	CAMERA	OBJECT	SCORE	ACTIONS
	aug. 27. 20:52:35	Ricsi regi	CAR	–	VIEW DELETE
	aug. 27. 20:34:53	Lia regi	PERSON	–	VIEW DELETE
	aug. 27. 20:34:47	Lia regi	PERSON	–	VIEW DELETE
	aug. 27. 20:34:18	Lia regi	PERSON	–	VIEW DELETE
	aug. 27. 20:33:46	Lia regi	PERSON	–	VIEW DELETE
	aug. 27. 20:33:22	Ricsi uj	FIRE	84%	VIEW DELETE
	aug. 27. 20:30:21	Ricsi uj	FIRE	86%	VIEW DELETE
	aug. 27. 20:29:33	Ricsi uj	FIRE	82%	VIEW DELETE
	aug. 27. 17:59:31	Lia regi	PERSON	–	VIEW DELETE
	aug. 27. 17:53:28	Lia regi	PERSON	–	VIEW DELETE

8. ábra: Az események böngészője. A találatok között a Frigate objektumdetekciói (person, car) és a tűzdetekciós mikroszolgáltatás találatai (fire) egységes listában jelennek meg, kamerára és objektumtípusra szűrhetően; a tűztalálatok mellett a modell pontszáma is megjelenik.

A 8. ábra a két eseményforrás egyesítését szemlélteti: a Frigate objektumdetektálásai és a saját tűzdetektor találatai ugyanabban a listában, azonos szerkezettel jelennek meg, holott két, egymást nem ismerő komponenstől származnak. A felhasználó számára a forrás különbsége nem látszik – ezt teszi lehetővé az MQTT-alapú szétcsatolás.

Ugyanezek a detektálások a beérkezés pillanatában, a kamerarács mellé nyitható oldalpanelen is megjelennek (9. ábra). A panel nem lekérdezésből dolgozik: a bejegyzések a WebSocket-kapcsolaton érkeznek, tehát a felhasználó akkor is látja az eseményt, amikor éppen az élőképet nézi.



9. ábra: Az élő eseményfolyam a kamerarács mellett. A panel a WebSocket-en érkező detektálásokat mutatja a beérkezés sorrendjében, kamerával, objektumtípussal és a detektor pontszámával.

A felvételek külön nézetben böngészhetők, kamerára és időtartományra szűrve (10. ábra); a lejátszás és a letöltés is a backend proxyján halad, tehát a böngésző soha nem lép közvetlen kapcsolatba a Frigate-tel.

CAMERA	STARTED	LENGTH	SIZE	ACTIONS
Lia regi	aug. 27. 20:34:53	1m 20s	–	PLAY DOWNLOAD DELETE
Lia regi	aug. 27. 20:34:47	1s	–	PLAY DOWNLOAD DELETE
Lia regi	aug. 27. 20:34:18	26s	–	PLAY DOWNLOAD DELETE
Lia regi	aug. 27. 20:33:46	33s	–	PLAY DOWNLOAD DELETE
Ricsi uj	aug. 27. 20:33:18	8s	–	PLAY DOWNLOAD DELETE
Lia regi	aug. 27. 20:31:36	28s	–	PLAY DOWNLOAD DELETE
Lia regi	aug. 27. 20:30:30	6s	–	PLAY DOWNLOAD DELETE
Ricsi uj	aug. 27. 20:30:11	40s	–	PLAY DOWNLOAD DELETE
Lia regi	aug. 27. 20:30:04	5s	–	PLAY DOWNLOAD DELETE
Lia regi	aug. 27. 20:29:37	12s	–	PLAY DOWNLOAD DELETE
Ricsi uj	aug. 27. 20:29:23	40s	–	PLAY DOWNLOAD DELETE
Ricsi regi	aug. 27. 20:28:57	1s	–	PLAY DOWNLOAD DELETE
Lia regi	aug. 27. 17:59:31	5s	–	PLAY DOWNLOAD DELETE
Lia regi	aug. 27. 17:53:28	4s	–	PLAY DOWNLOAD DELETE

10. ábra: A felvételek böngészője. A klipek kameránként és időtartományonként szűrhetők, közvetlenül lejátszhatók és letölthetők.

A kamerabeállítások nézet (11. ábra) a kiválasztott node paramétereit módosítja: a megjelenített nevet, a tájolást, a felbontást, a képsebességet, a képtükrözést és az aktív kameraszenzort. A beállítások egy része a telefonra megy, egyetlen kérdésben; a tájolás viszont a Frigate konfigurációjába kerül, a 3.5.4. szakaszban leírt okból.

Camera Settings 3/4 nodes up - 4 H.264 STREAMS 4 CAMS Testt LOGOUT

NODES

- Lia uj
- Ricsi uj
- Lia regi**
- Ricsi regi

Lia regi
cam_192_168_1_8

DISPLAY NAME
Lia regi

SAVE

ANDROID IP CAMERA SETTINGS

ORIENTATION: 90 RESOLUTION: 1280x720

FPS: 30

Mirror image: ☐ Front camera (FFC): ☐

SAVE SETTINGS

DANGER ZONE
The node is removed from the Frigate config and its alias is deleted. Recordings are kept.

DELETE NODE

ws connected
20:47:48
Testt

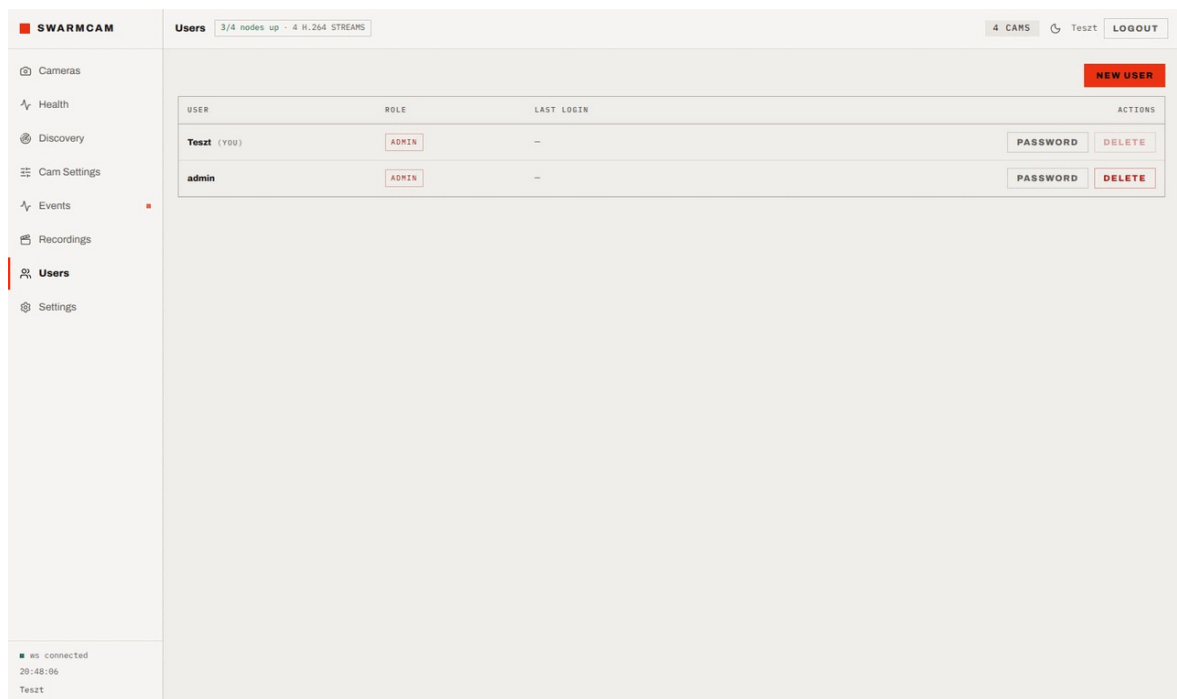
11. ábra: Kamerabeállítások. A bal oldali listából kiválasztott node paramétereit élőben módosíthatók; a „Danger zone” szekció a node eltávolítását teszi lehetővé, a hozzá tartozó felvételek megtartása mellett.

A rendszerbeállítások nézet (12. ábra) a Frigate globális konfigurációjának azon részeit teszi elérhetővé, amelyek az üzemeltetés során valóban változnak: a dekóder gyorsítási módját, az RTSP-átvitel protokollját, a detekciós képsebességet és felbontást, a követett objektumtípusokat és a felvételek megőrzési idejét. A mentés a konfigurációs fájlt írja, majd újraindítja a Frigate konténert – erre a felület külön figyelmeztet, mert az újraindítás alatt az élő folyamatok megszakadnak.

The screenshot displays the 'Settings' page of the SWARMCAM interface. The left sidebar contains navigation links: Cameras, Health, Discovery, Cam Settings, Events, Recordings, Users, and Settings (which is currently selected). The main content area is titled 'SETTINGS' and shows the 'Global Frigate configuration - docker/frigate/config.yml'. It is divided into several sections: 'DECODER' (with a dropdown for 'nvidia' under 'Hardware acceleration' and radio buttons for 'TCP' and 'UDP' under 'RTSP transport'), 'DETECTION' (with a dropdown for '5' under 'Detect FPS', a dropdown for '640x380' under 'Detect resolution', and checkboxes for 'person', 'cat', 'dog', 'bicycle', and 'motorcycle' under 'Tracked objects'), 'RETENTION' (with input fields for 'Clips (days)' set to '14' and 'Continuous (days)' set to '7'), and 'MQTT' (with a note about MQTT output). At the bottom, a message states 'Saving writes docker/frigate/config.yml and restarts Frigate - live streams may drop for a few seconds.' and a red 'SAVE CONFIG' button is present.

12. ábra: Rendszerbeállítások. A felület a Frigate konfigurációjának üzemeltetés közben ténylegesen változó részeit teszi elérhetővé; a mentés a konfigurációs fájl írása után újraindítja a Frigate konténert.

A felhasználókezelés (13. ábra) szintén proxyzott felület: a Frigate saját felhasználói adatbázisát kezeli, így a rendszernek nincs önálló felhasználónyilvántartása és nincs két, egymástól elcsúsztható jogosultsági modell sem.

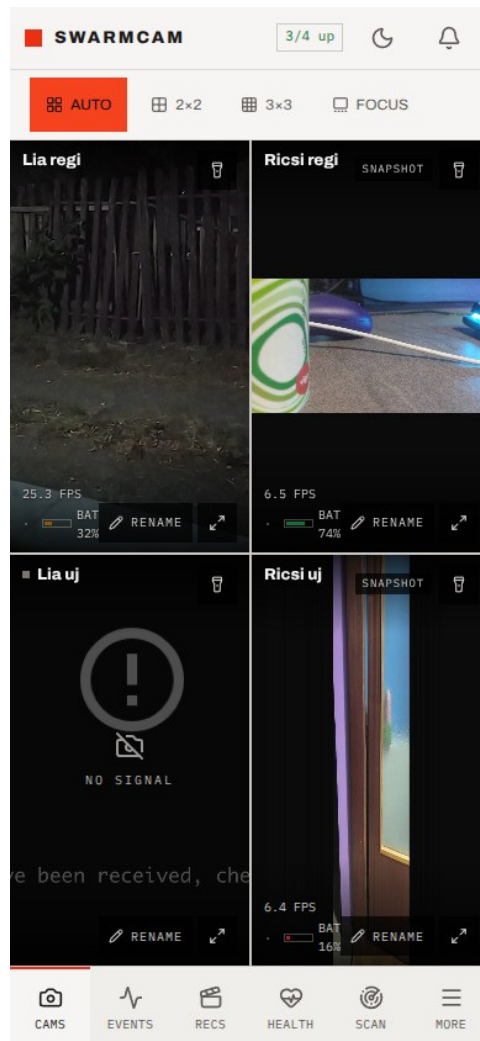


13. ábra: Felhasználókezelés. A rendszer nem tart fenn saját felhasználónyilvántartást: a Frigate felhasználói adatbázisát kezeli a backend proxyján keresztül.

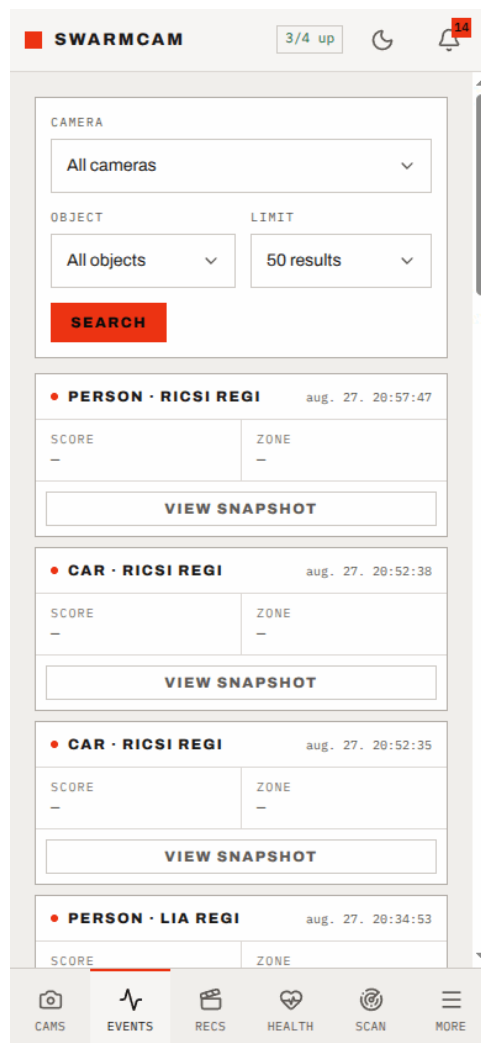
3.5.8. Mobilnézet

A felület reszponzív: a megjelenítés a rendelkezésre álló szélesség alapján két, határozottan eltérő elrendezés között vált – ez nem csupán oszlopszám-változtatás, hanem külön navigációs modell.

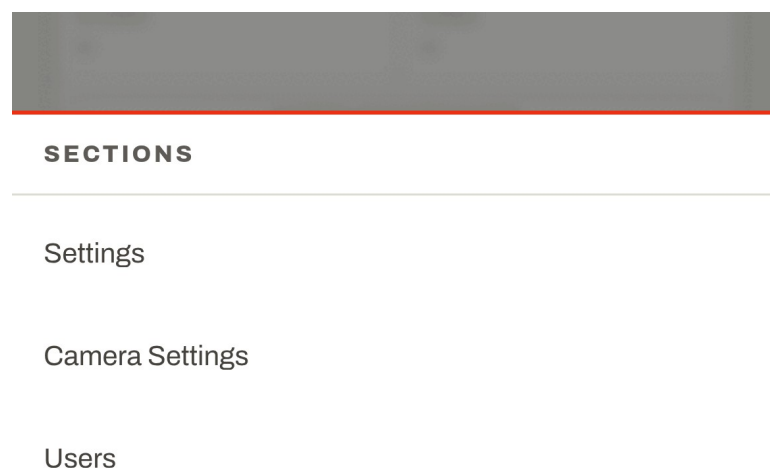
Asztali szélességen az oldalsáv állandóan látható, és a fejlécben elfér a kamerarács elrendezésválasztója, az eseménypanel kapcsolója és a felhasználói menü. Mobil szélességen az oldalsáv eltűnik: helyette rögzített fejléc és alsó fülsáv jelenik meg, amely a hat leggyakrabban használt nézetet hüvelykujjal elérhető távolságban tartja; a többi nézet egy alulról felcsúszó lapon érhető el. Az elrendezésválasztó megmarad, így a felhasználó eldöntheti, hogy több kamerát lát kisebb csempéken, vagy egyet a teljes szélességben.



14. ábra: A dashboard mobil elrendezése. Az oldalsáv helyét rögzített fejléc és a képernyő alján rögzített fűlsáv veszi át; a kamerarács a rendelkezésre álló szélességhez igazodik. A Ricsi regi és Ricsi uj node képén a fekete sávok oka a 3.5.4. szakaszban tárgyalt korlát: az álló tájolásra rögzített telefon álló képet illetve a fekvő képkockába, és ezt a szerveroldali forgatás már nem tudja visszanyerni.



15. ábra: Az események böngészője mobil elrendezésben. Az asztali táblázat kártyás nézetű alakul, hogy vízszintes görgetés nélkül olvasható maradjon.



16. ábra: A „More” lap mobil nézetben. A fűlsávba nem férő nézetek egy alulról felcsúszó lapon érhetők el. A háttér elhomályosítása a felület része, nem a képernyőkép hibája: a felcsúszó lap mögött scrim és háttérelmosás fut, hogy a figyelem a menüre essen.

A 14. ábra a kamerarácsot, a 15. ábra az eseményböngészőt mutatja mobil elrendezésben; utóbbin jól látszik, hogy az asztali táblázatból kártyalista lesz, mert egy hatoszlopos táblázat telefonon vízszintes görgetés nélkül olvashatatlan volna. A fülsávba nem férő nézetek a 16. ábrán látható, alulról felcsúszó lapon érhetők el.

A mobilnézet létjogosultsága gyakorlati: egy otthoni biztonsági rendszert tipikusan telefonon néz meg a felhasználó. Az élőkép itt ugyanazon az MSE-csatornán fut, mint az asztali változatban.

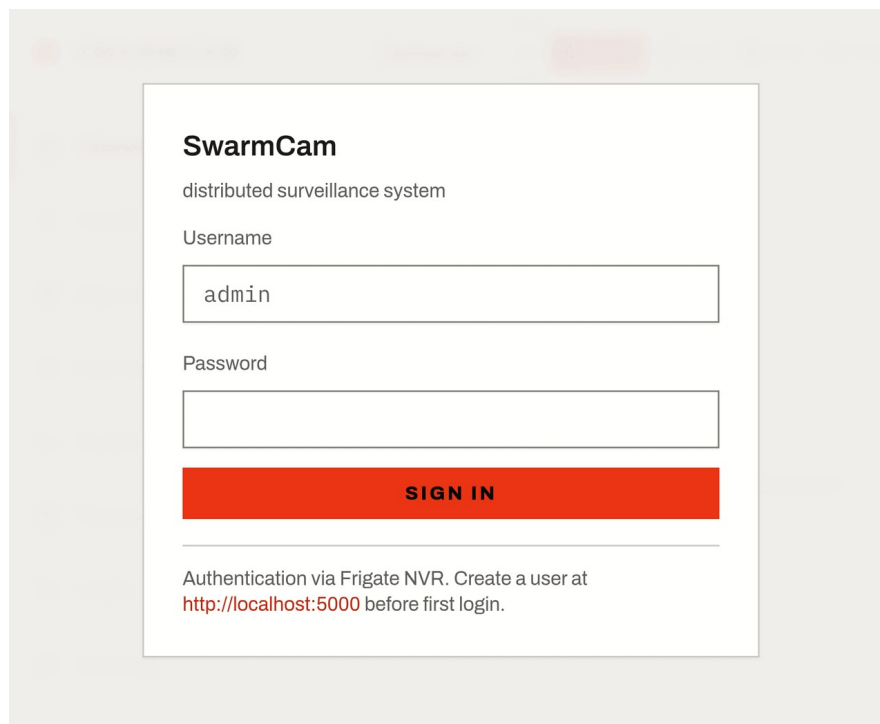
3.5.9. Hitelesítés és biztonsági megfontolások

A hitelesítés a Frigate felhasználói adatbázisára épül: a bejelentkezési űrlapon (17. ábra) megadott adatokat a backend továbbítja a Frigate bejelentkezési végpontjának, majd a kapott JWT-t [28] adja vissza a felületnek. Minden további kérés ezt a tokent hordozza.

A frontend a tokent kizárólag a React alkalmazás memóriájában tárolja, nem a böngésző tartós tárolójában. A tartós tárolóban elhelyezett token ugyanis minden, az oldalon futó szkript számára olvasható, tehát egy sikeres szkriptbeillesztéses támadás azonnal átvehető munkamenetet adna. Ára, hogy a token az oldal újratöltésekor elvész – egy helyi adminisztrációs felületen ez elfogadható.

A backend ezen felül saját hitelesített munkamenetet is fenntart a Frigate felé, mert az állapotfigyelés, a statisztikák lekérése és a konfiguráció olvasása felhasználói beavatkozás nélkül, folyamatosan zajlik. A két munkamenet független: a felhasználói token lejárta nem érinti a háttérfolyamatokat, a háttérmunkamenet megújítása pedig a bejelentkezett felhasználót.

A hibakezelés két státuszkódot különböztet meg élesen. A 401-es válasz – érvénytelen vagy lejárt token – kijelentkezteti a felhasználót; az 503-as – a Frigate éppen újraindul egy konfigurációmentés után – viszont nem, ilyenkor a felület hibaüzenetet mutat, de a munkamenetet megtartja. A két eset összemosása azt eredményezné, hogy minden konfigurációmentés kijelentkeztetne.



The image shows a web-based login interface for SwarmCam. It features a white login box on a light gray background. The box has the title 'SwarmCam' and the subtitle 'distributed surveillance system'. Below this, there are two input fields: 'Username' with the value 'admin' and 'Password' which is empty. A red 'SIGN IN' button is positioned below the password field. At the bottom of the box, there is a note: 'Authentication via Frigate NVR. Create a user at <http://localhost:5000> before first login.'

17. ábra: Bejelentkezési felület. A hitelesítés a Frigate felhasználói adatbázisára épül; a kapott JWT kizárólag az alkalmazás memóriájában él, tartós böngészőtárolóba nem kerül.

Végül a biztonsági modell határa: a SwarmCam kizárólag megbízható, helyi hálózaton üzemelő rendszernek készült. Az MQTT-bróker névtelen hozzáférést enged, a kamerák ön aláírt tanúsítványát a rendszer ellenőrzés nélkül elfogadja, a konténerek nagy része a gazdagép hálózati névterét használja, és a backend hozzáfér a Docker-sockethez. Ezek a döntések a helyi hálózat feltételezésével együtt védhetők; internet felőli kitettség esetén viszont mindegyik önálló sebezhetőséggé válna. Távoli eléréshez a helyes megoldás a rendszer elé helyezett VPN, nem a portátírányítás.

3.6. Tesztelés és mérési eredmények

A rendszer valós otthoni hálózati környezetben, több különböző generációjú Android eszközzel került tesztelésre, és a fejlesztés teljes ideje alatt éles, folyamatos üzemben működött. Ez a szakasz a számszerű mérések eredményeit közli.

3.6.1. A mérési környezet és a módszertan

A méréseket a 7. táblázatban leírt gépen végeztem, a teljes rendszer szokásos, konténerizált konfigurációjában.

7. táblázat: A mérési környezet

Elem	Érték
Gép	ASUS ROG Strix G513QM hordozható munkaállomás
Processzor	AMD Ryzen 9 5900HX, 8 mag / 16 szál
Memória	15 GiB
GPU	NVIDIA GeForce RTX 3060 Laptop, 6 GiB, 595.84 illesztőprogram
Operációs rendszer	Ubuntu 24.04.3 LTS
Konténerkörnyezet	Docker 28.5.2, Docker Compose v2
Frigate	0.17.2, stable-tensorrt image
Detektor	ONNX Runtime, yolo-generic modell, 320×320 bemenet, COCO-80 címkekészlet
Backend	Python 3.12, FastAPI 0.141
Kamera-node	Android IP Camera, 1280×720, H.264, HTTPS

A mérések során felmerült egy módszertani nehézség. A kameraszám függvényében vizsgált erőforrás-használathoz és a detektor összehasonlító méréséhez minden mérési pontban azonos bemenetre van szükség, különben a mérési pontok különbsége nem a vizsgált változóból ered. Élő telefonokkal ez nem biztosítható: a jelenet, a megvilágítás és így a képsebesség percről percre változik.

A megoldás az volt, hogy a mérés bemenete egy korábban, ugyanezekről a telefonokról rögzített, valós H.264 felvétel: 1280×720 felbontású, 10 kép/másodperces állomány, amelyet a Frigate minden mérési pontban ugyanúgy, N különálló kameraként dolgoz fel. Így minden mérési pont pontosan azonos bemenetet kap, a mérés pedig bármikor megismételhető, éjjel-nappal, a telefonok állapotától függetlenül.

A módszer egyetlen eltérése az éles üzemtől, hogy ebben az elrendezésben a Frigate közvetlenül olvassa a felvételt, a go2rtc újracsomagoló lépése nélkül. A go2rtc itt nem kódol újra, csak konténerformátumot vált, így a hozzájárulása kicsi és állandó.

A mérőszkript (benchmark/measure.py) minden mérési pontban hatvan másodpercen át, kétmásodpercenként mintavételezi a konténerek erőforrás-használatát, és a minták átlagát, maximumát és darabszámát rögzíti; a mérési pontok között a rendszer hatvan másodpercet stabilizálódik.

3.6.2. Erőforrás-használat a kameraszám függvényében

Az első mérés azt vizsgálja, hogyan skálázódik a rendszer erőforrásigénye a bekötött kamerák számával. A mérés nulla kamerával indul – ez adja a rendszer alapterhelését –, majd egytől négy kameráig halad, mérési pontonként hatvan másodperces mintavétellel.

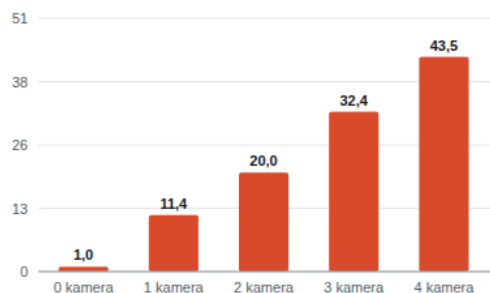
8. táblázat: A konténerek erőforrás-használata a kameraszám függvényében (60 s alatt, 2 s-onként vett minták átlaga)

Kameraszám	0	1	2	3	4
frigate — CPU (%)	1,0	11,4	20,0	32,4	43,5
frigate — memória (MiB)	851	1414	1521	1632	1725
backend — CPU (%)	0,40	0,56	0,45	0,61	0,55
backend — memória (MiB)	48	49	50	52	52
fire-detector — CPU (%)	0,40	0,27	0,36	0,33	0,32
fire-detector — memória (MiB)	131	131	131	131	131
mosquitto — CPU (%)	0,04	0,04	0,03	0,03	0,03
dashboard — CPU (%)	0,00	0,01	0,01	0,01	0,01

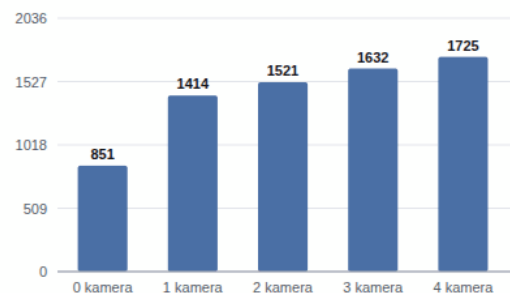
A mért értékeket a 8. táblázat, a Frigate konténer skálázódását pedig a 18. ábra foglalja össze. Az adatokból három következtetés vonható le.

Erőforrás-használat a kameraszám függvényében

FRIGATE — PROCESSZORHASZNÁLAT (%)



FRIGATE — MEMÓRIA (MiB)



18. ábra: A Frigate konténer processzor- és memóriahasználata a bekötött kamerák számának függvényében. A processzorterhelés kameránként közel állandó értékkel nő, a memóriaigény pedig az első kamera után gyakorlatilag telítődik.

Az első, hogy a rendszer alapterhelése elhanyagolható: kamerák nélkül a Frigate processzorhasználat 1 százalék alatt marad; ami mérhető terhelés, az szinte teljes egészében memória – a betöltött modell és a futtatókörnyezet önmagában 851 MiB-ot foglal.

A második, hogy a processzorterhelés a kameraszámmal lényegében lineárisan nő: 1 kameránál 11,4%, kettőnél 20,0%, háromnál 32,4%, négyenél 43,5%. A növekmény kameránként 10,6 százalékpont körül állandó, tehát a rendszer erőforrásigénye előre becsülhető: minden további kamera nagyjából ugyanannyiba kerül. Ez a Frigate felépítéséből következik – kameránként külön dekóder- és detekciós folyamat fut –, és egyben azt is jelzi, hogy a mérési tartományban nincs olyan közös erőforrás, amely torlódást okozna.

A harmadik, hogy a többi konténer terhelése a kameraszámtól gyakorlatilag független: a backend processzorhasználat minden mérési pontban 1 százalék alatt marad, memóriaigénye 48–50 MiB, a bróker és a webkiszolgáló pedig mérhetetlenül keveset fogyaszt. A rendszer erőforrásigénye tehát egyetlen komponensen, a videófeldolgozáson múlik.

A gyakorlati üzemeltetésre nézve a 43,5 százalékos érték a docker stats konvenciója szerint egyetlen processzormag terheléséhez viszonyítva értendő, tehát négy kamera a tizenhat szálú gép kapacitásának alig 2,7 százalékát köti le. A rendszer szűk keresztmetszete ezen a ponton nem a processzor, hanem a detektor átbocsátóképessége és a lemez írási sávszélessége.

3.6.3. Detektálási sebesség: GPU-gyorsítás és processzoros futtatás

A második mérés a hardveres gyorsítás hatását vizsgálja a detektálási sebességre. Az összehasonlítás tisztaságát az biztosítja, hogy a két futtatás mindenben megegyezik – azonos modell, bemenet, kameraszám és detekciós felbontás –, és kizárólag a detektor futtatási eszköze tér el.

A mérés egy további beavatkozást igényelt: az alapbeállítás szerinti mozgásmaszk mellett a detektor túl ritkán szólal meg ahhoz, hogy a következtetési idő megbízhatóan mérhető legyen, ezért a mérés idejére a mozgásküszöb a lehető legalacsonyabbra került, hogy a detektor gyakorlatilag minden képkockán lefusson.

A mérést egy másik felvételrészletből épített bemenettel meg is ismételttem. A következtetési idők nagyságrendje változatlan maradt – GPU-n 7,4, processzoron 36,4 ezredmásodperc –, a két eszköz aránya viszont 4,9-szeres lett a 9. táblázatban közölt 7,8-szeres helyett, mert a telítetlen futáson a GPU-s következtetési idő volt a magasabb. A második futás bemenetén ugyanis lényegesen kevesebb mozgás volt, így a detektor nem telítődött, és a másodpercenkénti detektálásszám nem jellemezte a kapacitást. A táblázat ezért a telített futás adatait közli; mindkét futás nyers adatai megtalálhatók az elektronikus mellékletben.

9. táblázat: A detektor következtetési ideje GPU-gyorsítással és processzoron (azonos modell, azonos bemenet, 3 kamera, telített detektor)

Mutató	GPU (ONNX Runtime, CUDA)	Processzor (ONNX Runtime, CPU)
Következtetési idő képkockánként (ms)	5,54	43,12
Legnagyobb mért következtetési idő (ms)	5,57	47,07
Összesített detekciós képsebesség (kép/s)	15,68	14,07
Elméleti detekciós felső korlát (kép/s)	180,5	23,2

A 9. táblázat szerint a GPU-gyorsítás a képkockánkénti következtetési időt 43,12 ezredmásodpercről 5,54 ezredmásodpercre csökkenti – ez 7,8-szeres gyorsulás, azonos modellel és azonos bemenettel. Három értelmezési megjegyzés kívánczik mellé.

Az első, hogy a processzoros futtatás nem használhatatlan – csak a kiszolgálható kameraszámot korlátozza. A detektor egyetlen, sorosított erőforrás: ha egy képkocka feldolgozása n ezredmásodperc, akkor a rendszer másodpercenként legfeljebb $1000/n$ detektálást tud elvégezni, függetlenül attól, hány kamera kéri, és ez az érték oszlik szét a kamerák között. A mért értékekből ez processzoron 23,2, GPU-n 180,5 detektálás másodpercenként.

A második, hogy ezt a korlátot a mérés éppen elérte, de nem lépte túl: három kamera 5 kép/másodperces detekcióval 15 detektálást igényel másodpercenként, ebből a processzoros futtatás 14,07-et, a GPU-s 15,68-at teljesített. A 23,2-es elméleti korlát azt is megmutatja, hogy a processzor négy-öt kameránál telítődne, míg a GPU ugyanezen a beállításon több mint harminc kamerát is kiszolgálna.

A harmadik, hogy a mérési gép processzora tizenhat szásas, tehát viszonylag erős. Egy tipikus, alacsony fogyasztású mini PC-n a processzoros érték ennél lényegesen rosszabb lenne, ami a GPU – vagy alternatívaként egy Coral típusú célhardver – melletti érvet tovább erősíti.

A mérés gyakorlati tanulsága, hogy az alapértelmezett konfiguráció szándékosan processzoros detektorral indul – az minden gépen működik –, a GPU-gyorsítás pedig a beállítások felületén, egyetlen legördülő menüből kapcsolható be.

3.6.4. Eseménykésleltetés és az állapotfigyelés pontossága

A harmadik mérés a rendszer valós idejű viselkedését vizsgálja két mutatón: az állapotfigyelő hurok periódusának tényleges szórásán, és a rendszer saját eseményátjátszási késleltetésén – azon, hogy mennyi idő telik el az esemény brókerre érkezésétől a böngészőben nyitott WebSocket-kapcsolaton történő megérkezéséig.

A második mutató méréséhez szintetikus, a Frigate üzenetformátumával megegyező eseményeket publikáltam a brókerre. Ez fontos módszertani pont: a mért érték kizárólag a rendszer saját átjátszási láncát – bróker, backend feldolgozás, WebSocket-továbbítás – jellemzi, és nem tartalmazza a Frigate detektálási idejét. Ez a szétválasztás szándékos: a detektálási időt a 3.6.3. szakasz méri külön, az átjátszási lánc pedig az, amiért a jelen munka felelős.

10. táblázat: Az állapotfigyelő hurok periódusa és az MQTT → WebSocket átjátszási késleltetés

Mutató	Érték
Az állapotfigyelő hurok átlagos periódusa (s)	7,842
A periódus szórása (s)	1,945
Átjátszási késleltetés — átlag (ms)	0,7
Átjátszási késleltetés — 95. percentilis (ms)	1,2
Illesztett eseménypárok száma	29
A mérés hossza (s)	120

Az eredmény a 10. táblázatban látható. Az átjátszási lánc átlagos késleltetése 0,7 ezredmásodperc, a 95. percentilis 1,2 ezredmásodperc; a harminc publikált eseményből huszonkilenc érkezett meg a WebSocket-kliensig, az egyetlen illesztetlen esemény a mérési ablak határára esett. Ez a nagyságrend a felületen érzékelhetetlen, és visszaigazolja a push-alapú közvetítés melletti döntést: egy egy másodperces periódusú lekérdezés ugyanezt átlagosan félmásodpercre növelte volna, ráadásul üresjárat terhelés árán.

Az állapotfigyelő hurok mért periódusa 7,84 másodperc, 1,95 másodperces szórással, négy nyilvántartott node mellett. Az öt másodperces névleges értéket tehát átlagosan 2,84 másodperccel lépi túl, ami a kamerák lekérdezési körének ideje: a hurok sorosan kérdezi le a

node-okat, majd a teljes kör után vár öt másodpercet, így a tényleges periódus a névleges érték és a kör hosszának összege.

A többlet szinte teljes egészében egyetlen forrásból származik: a mérés idején a négy node egyike nem válaszolt, és annak lekérdezése minden körben kimerítette a kétmásodperces időkorlátot, míg a három elérhető node együtt is néhány száz ezredmásodperc alatt válaszolt. Ez magyarázza a szórást is, és egyben egy tervezési korlátot tesz számszerűvé: a soros lekérdezés a nem elérhető node-ok számával rontja a hurok periódusát, tehát az állapotváltozás felismerésének késleltetése is ezzel arányosan nő. A javítás iránya a lekérdezés párhuzamosítása; a jelenlegi kameraszámnál viszont nem jelent szűk keresztmetszetet.

3.6.5. Az élőkép architektúrájának mérése

A negyedik mérés a 3.5.3. szakaszban ismertetett architektúraváltást igazolja, ugyanazon a rendszeren, ugyanazokkal a kamerákkal, egyetlen néző mellett.

11. táblázat: Az élőkép két megvalósításának összehasonlítása

Szempont	MJPEG-proxy (korábbi)	go2rtc MSE (jelenlegi)
Megjelenített felbontás	640×360 (a detekciós méret)	1280×720 (a telefon natív mérete)
Képfrissítés	3,5 kép/s (a detekciós képsebességhez kötve)	A telefon képsebessége, a detekciótól függetlenül
Frigate processzorterhelése nézőnként	+26 százalékpont	+2 százalékpont
Végponttól végpontig mért késleltetés	Nem mérve; szemmel láthatóan magasabb	0,03–0,07 s
Szerveroldali újrakódolás	JPEG-kódolás nézőnként	Nincs, csak konténerformátum-váltás

A 11. táblázat legfontosabb sora a processzorterhelés: a korábbi megoldás minden egyes megnyitott kamerakártyáért egy teljes JPEG-kódolót futtatott, tehát négy egyszerre nézett kamera önmagában több mint egy processzormagnyi terhelést jelentett – ráadásul azért, hogy a felhasználó a rögzítettől rosszabb minőségű képet lásson. A jelenlegi megoldásban a szerver nem kódol, csak továbbít, így a nézők száma gyakorlatilag nem befolyásolja a szerver terhelését.

3.6.6. A képsebesség fényfüggése – a telefonoldali korlát

Az ötödik mérés a rendszer egyik legfontosabb, ugyanakkor a legkevésbé nyilvánvaló korlátját dokumentálja. Az élőkép képsebessége nem a szervertől, nem a hálózattól és nem a beállított értéktől függ, hanem a kamera előtti megvilágítástól.

Az Android automatikus expozíciója [29] gyenge fényben megnöveli az expozíciós időt, hogy a kép ne legyen alulexponált. A hosszabb expozíció viszont fizikai korlátot szab a másodpercenkénti képkockaszámnak: ha egy képkocka rögzítése 300 ezredmásodpercig tart, másodpercenként legfeljebb három képkocka állítható elő – függetlenül attól, mit kér a szerver.

A jelenség egy node zseblámpájának ki- és bekapcsolásával reprodukálható, mert a rendszer a valós, mért képsebességet a kamerakártyán megjeleníti: sötétben 3,3 kép/másodperc, a zseblámpa bekapcsolása után – változatlan jelenet és szerveroldali beállítások mellett – 11,3 kép/másodperc.



19. ábra: A képsebesség fényfüggése ugyanazon a node-on. A kártya bal alsó sarkában megjelenő, ténylegesen mért képsebesség sötétben 3,3 kép/s, a zseblámpa bekapcsolása után 11,3 kép/s – a jelenet és minden szerveroldali beállítás változatlan.

A 19. ábra ugyanazt a kamerát mutatja a zseblámpa kikapcsolt és bekapcsolt állapotában. A következtetés egyértelmű: ez a korlát szerveroldalról nem kezelhető, mert a képkockák elő sem állnak. Aki telefonalapú kamerarendszert épít, annak a megvilágításról külön kell gondoskodnia – éjszakai üzemből ez a rendszer legfontosabb gyakorlati korlátja.

Ugyanez a hatás magyarázza, miért fontos, hogy a rendszer a *ténylegesen mért* képsebességet jelenítse meg a beállított érték helyett. A beállított érték a felhasználót félrevezetné: azt sugallná, hogy a rendszer huszonöt képet ad másodpercenként, miközben ténylegesen hármat.

3.6.7. Megfigyelt hibamódok

A folyamatos, éles üzemű tesztelés több olyan hibamódot is felszínre hozott, amelyek megléte a rendszer megbízhatóságának reális megítéléséhez tartozik.

A leggyakoribb hibamód a kamera-node váratlan elérhetetlensége. Ez a rendszer szempontjából nem hiba, hanem üzemszerű állapot: az állapotfigyelő hurok néhány másodpercen belül észleli, a felület egyértelműen jelzi, a többi node pedig zavartalanul működik tovább.

A második, alattomosabb hibamód a telefon videókódolójának elakadása. Ilyenkor a készülék hálózatilag elérhető marad – az állapotleíró válaszol, a node „online” –, a H.264 végpont viszont üres választ ad, és a Frigate ffmpeg-folyamata újraindulási hurokba kerül. A jelenlegi állapotfigyelés ezt nem ismeri fel, mert az elérhetőséget az állapotleíróval méri, nem a videófolyammal; a felület közvetve mégis jelzi, mert a kamerakártya képsebessége nullára esik. A megbízható felismeréshez a folyam meglétét is figyelni kellene.

A harmadik hibamód a tűzdetektor téves riasztása gyenge fényben: a modell éjszakai, erősen zajos képeken füstöt jelez ott, ahol nincs. Az ok feltehetően az, hogy a nagy erősítéssel készült képen a szenzoraj textúrája statisztikailag hasonlít arra, amit a modell füstként tanult meg.

A hibamód a küszöb megemelésével kezelhető volt. A detektor eseménynaplójában rögzített találatok átnézése azt mutatta, hogy az eredeti ötven és a nyolcvan százalék közötti sávba eső detektálások lényegében kivétel nélkül tévesek voltak, míg a valódi lángra adott találatok pontszáma – a mért esetekben 82 és 86 százalék között – ezt a sávot meghaladta. A küszöb nyolcvan százalékra emelése után a téves éjszakai riasztások megszűntek, valódi találat elvesztése nélkül. Ez a küszöb a jelen telepítés kameráira illeszkedik, általános érvényűnek nem tekinthető; a robusztusabb megoldás a képfényesség alapján váltó, külön éjszakai küszöb volna.

3.7. A megoldás korlátai

Az alábbi korlátok mindegyike tudatosan vállalt kompromisszum vagy azonosított hiányosság, nem pedig felderítetlen hiba.

A biztonsági modell a megbízható helyi hálózat feltételezésére épül – a 3.5.9. szakaszban felsorolt döntések mindegyike ebből ered, és a rendszer közvetlen internetes kitettsége mindegyiket sebezhetőséggé változtatná.

A rendszer állapota a memóriában él. A kameralista minden induláskor a Frigate konfigurációjából épül újra, ami egyetlen csomópontra és kis kameraszámra megfelelő, de kizárja a többcsomópontos üzemet és a hosszabb távú állapottörténet vezetését.

A kamera-node megbízhatósága a telefontól függ, és a rendszer csak azt látja belőle, amit a rajta futó alkalmazás megmutat: az Android energiagazdálkodása leállíthatja az alkalmazást,

a videókódoló elakadhat, távoli helyreállításra nincs mód, a protokollcserével pedig a töltési állapot jelzése is elveszett.

A tűzdetekciós modell nem saját betanítású, hanem nyilvánosan elérhető súlykészlet, tehát viselkedése nem a jelen telepítés körülményeire optimalizált – ez okozza a gyenge fényben tapasztalt téves riasztásokat.

Végül a rendszer jelenleg a telefon szenzorai közül gyakorlatilag csak a kamerát és az akkumulátor-, illetve Wi-Fi-állapotot használja. A mikrofon, a gyorsulásmérő és a giroszkóp mint eseményforrás nincs bevonva, holott az architektúra – az MQTT-alapú, szétcsatolt eseményközvetítés miatt – ezek fogadására különösebb átalakítás nélkül alkalmas lenne.

3.8. Tapasztalatok és tanulságok

A rendszer minden lényeges eleme elkészült és éles üzemben működik, a fejlesztés során szerzett tapasztalatok viszont önmagukban is a munka eredményéhez tartoznak. Három olyan tanulság van, amely a következő hasonló projektemet biztosan meghatározza.

Az első, hogy a kamera-node fekete doboz maradt. Egy kész, idegen alkalmazásra épülő node esetén nem én döntöm el, milyen adat érhető el róla és milyen formában: a protokollcserével elveszett a töltési állapot, a videókódoló elakadását pedig azért nem tudom megbízhatóan felismerni, mert az alkalmazás elérhetőnek mutatja magát akkor is, amikor már nem ad képet. A távoli újraindításra sincs mód, mert az alkalmazás nem kínál rá felületet. Ezek nem megvalósítási hibák, hanem annak a döntésnek a következményei, hogy a node-oldalt készen kaptam.

A második, hogy minden újrakódolást végző réteg rejtett költséget hoz magával, és ez a költség jellemzően nem ott jelentkezik, ahol az ember keresi. A forgatás miatt beiktatott átkódolás az időbélyegek elvesztésén keresztül a detekciót állította le úgy, hogy közben a felvétel zavartalanul készült; a felderítésből érkező, rossz felbontásérték pedig a mozgásdetektálást skálázta fel akkora képre, hogy egyetlen kamera közel háromszáz százalék processzoridőt evett. Mindkét hiba kifelé alig látszott, és mindkettőt mérés hozta felszínre, nem szemrevételezés.

A harmadik, hogy a mérés a tervezés része, nem az utólagos igazolásé. Az élőkép architektúrájának cseréje, a detektor GPU-ra terelése és a tűzriasztási küszöb megválasztása egyaránt olyan döntés volt, amelyet szemre nem lehetett eldönteni; a végleges értékeket minden esetben a mért adat adta.

Ha ma kezdeném a munkát, a kamera-node oldalát magam írnám meg, és a saját alkalmazás két dolgot csinálna másképp. Egyrészt nem nyers H.264 elemi folyamat adna HTTP-n, hanem szabványos RTSP-kiszolgálót futtatna: a folyam így saját, helyes időbélyegekkal érkezne, ami egyben feleslegessé tenné az időbélyeg-pótló ffmpeg-presetet és a nyers folyam RTSP-vé csomagolását is. Másrészt a képet már a kódolás előtt, a telefon oldalán forgatná el, a rögzítési lánc részeként – ekkor a szerverre eleve a kívánt tájolású, teljes felbontású kép

érkezne, és a 3.5.4. szakaszban tárgyalt teljes problémakör – a szerveroldali újrakódolás, annak processzorköltsége és az időbélyegek elvesztése – meg sem jelenne. Ehhez járulna a teljes telemetria a töltési állapottal együtt, valamint egy távolról vezérelhető felügyeleti funkció, amely a beragadt kódot újraindítja; ez a fennmaradó hibamódok többségét is megszüntetné.

A választás mégis tudatos volt: egy kész, nyílt forráskódú alkalmazásra építve a rendszer bármely telefonon, saját alkalmazás terjesztése és karbantartása nélkül használható, a munka súlypontja pedig a szerveroldali integráción maradhatott – és ezt az irányt szerettem volna végigvinni.

4. Összegzés

A szakdolgozat célja egy felhőfüggetlen, lokális hálózaton működő, mesterséges intelligenciával támogatott kamerarendszer (SwarmCam) tervezése és megvalósítása volt, amely feleslegessé vált Android okostelefonokat integrál dedikált kamerahardver nélkül. A megvalósítás igazolta, hogy egy hálózati felderítésre, konténerizált mikroszolgáltatásokra (Frigate, saját FastAPI backend, önálló YOLO-alapú tűzdetektor) és eseményvezérelt kommunikációra (MQTT, WebSocket) épülő architektúra alkalmas ilyen heterogén, régi eszközökből álló megfigyelőhálózat kiszolgálására. A felderítés azt a lépést váltja ki, amely a hagyományos NVR-eknél kameránként kézi munkát jelent: a node címét, elérési útját és paramétereit nem kell a rögzítő konfigurációjába írni. Ez azonban nem konfigurációmentes üzemet jelent. A node-oldali beállítás – az alkalmazás telepítése, a felbontás, a képsebesség és különösen a tájolás beállítása – megmarad, és készülékenként eltérő finomhangolást kíván: a kamera orientációja nem minden telefonon kezelhető ugyanúgy, a szerveroldali forgatás pedig külön feldolgozási ágat és processzorköltséget hoz magával. A rendszer tehát a kamerák bekötésének kézi konfigurációját szünteti meg, nem a telepítés és a hangolás egészét.

A kitűzött részcélok – az automatikus felderítés, a felhőfüggetlen szerverkörnyezet, az AI-alapú detekció (objektum-, valamint tűz-/füstfelismerés), a valós idejű eseménykezelés és a kezelőfelület – megvalósultak. A rendszer moduláris felépítése (különálló Docker konténerek, jól definiált MQTT/WebSocket interfészek) lehetővé teszi az egyes komponensek önálló továbbfejlesztését anélkül, hogy a többi részt érintené.

A dolgozatban bemutatott rendszer saját hozzájárulást jelentő komponenseit – a hálózati felderítést és az ahhoz tartozó ujjlenyomat-vizsgálatot, a FastAPI backendet és annak valós idejű kommunikációs rétegét (állapotfigyelő hurok, MQTT-átjátszás, WebSocket-elosztás), az élőkép-architektúrát a go2rtc MSE-relayjével, a kameraforgatás feldolgozási láncát, a tűz-/füstdetekciós mikroszolgáltatás teljes Python-pipeline-ját, valamint a React alapú, asztali és mobil elrendezést egyaránt kiszolgáló kezelőfelületet – magam terveztem és valósítottam meg, a 3. fejezetben indokolt tervezési döntésekkel és a 3.6. szakasz mérési módszertanával együtt. A rendszer emellett több készen kapott komponensre épül: a Frigate NVR-re és a benne futó go2rtc-re, az Android IP Camera alkalmazásra, a Mosquitto brókerre, valamint a tűz-/füstdetekcióban használt, előre betanított YOLO-modellre (yolov26-fire-detection [10]) – ez utóbbit nem én tanítottam be, csak a saját pipeline-omba integráltam.

4.1. Jövőbeli fejlesztési lehetőségek

A jelenlegi, működő rendszer több irányban is bővíthető:

- ONVIF-kompatibilis, dedikált IP kamerák (pl. PTZ funkcióval) bevonása a hálózatba, a discovery modul kiterjesztésével.
- A tűz-/füstdetekciós modell további finomhangolása (transfer learning) saját gyűjtésű adathalmazon, a téves riasztások arányának csökkentésére.
- Push értesítést támogató natív mobilalkalmazás fejlesztése a webes dashboard mellé, amely háttérben futva is riasztást tud küldeni.
- A Google Coral TPU-alapú gyorsítás kiértékelése alternatívaként az NVIDIA GPU mellett, alacsonyabb energiafogyasztású telepítésekhez.
- A telefonok további beépített szenzorainak (mikrofon, gyorsulásmérő, giroszkóp) bevonása újabb eseményforrásként, a jelenleg is felhasznált kamerakép, akkumulátorszint és Wi-Fi jelerősség mellé.
- A kamera-node videókódolójának elakadását felismerő figyelőmechanizmus: a rendszer jelenleg „online” állapotúnak mutatja azt a node-ot is, amely hálózatilag válaszol, de videót már nem szolgáltat.
- A tűz-/füstdetektor fényviszony-érzékeny küszöbölése, hogy az éjszakai, erősen zajos képeken tapasztalt téves füstriasztások megszűnjenek.

A dolgozatban bemutatott rendszer fejlesztése során mesterséges intelligencián alapuló eszközöket (Claude, Gemini) használtam: a forráskód egyes részeinek megfogalmazásához és átnézéséhez, a hibakereséshez, valamint a konfigurációs és üzemeltetési lépések ellenőrzéséhez.

Ugyanezeket az eszközöket a dolgozat szövegének elkészítéséhez is igénybe vettem: a szakirodalom kereséséhez és rendszerezéséhez, a szöveg nyelvi gondozásához és a fejezetszerkezet ellenőrzéséhez. Minden megoldást valós üzemi környezetben ellenőriztem; a dolgozatban szereplő tartalmak, mérési eredmények és következtetések a saját munkám eredményei, amelyekért a teljes felelősséget vállalom.

Irodalom

- [1] Frigate NVR – Documentation. [Online]. Elérhető: <https://docs.frigate.video/>
- [2] OASIS, MQTT Version 5.0 – OASIS Standard. [Online]. Elérhető: <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>
- [3] ONVIF, ONVIF Core Specification. [Online]. Elérhető: <https://www.onvif.org/specs/>
- [4] P. Khlebovich, IP Webcam. [Mobilalkalmazás]. Google Play Áruház.
- [5] I. Goodfellow, Y. Bengio, A. Courville, Deep Learning. Cambridge, MA: MIT Press, 2016.
- [6] Ultralytics, YOLO Docs. [Online]. Elérhető: <https://docs.ultralytics.com/>
- [7] V. Forti, C. P. Baldé, R. Kuehr, G. Bel, The Global E-waste Monitor 2020. Bonn/Geneva/Rotterdam: United Nations University/UNITAR, ITU, ISWA, 2020.
- [8] FastAPI – Documentation. [Online]. Elérhető: <https://fastapi.tiangolo.com/>
- [9] Docker, Docker Compose – Documentation. [Online]. Elérhető: <https://docs.docker.com/compose/>
- [10] S. Al-Haismawi, yolov26-fire-detection [Betanított modell]. Hugging Face. [Online]. Elérhető: <https://huggingface.co/SalahALHaismawi/yolov26-fire-detection>
- [11] React – Documentation. [Online]. Elérhető: <https://react.dev/>
- [12] DigitallyRefined, Android IP Camera. [Mobilalkalmazás]. F-Droid. [Online]. Elérhető: <https://github.com/DigitallyRefined/android-ip-camera>
- [13] Microsoft, ONNX Runtime – Documentation. [Online]. Elérhető: <https://onnxruntime.ai/docs/>
- [14] A. Ivanov, go2rtc – Ultimate camera streaming application. [Online]. Elérhető: <https://github.com/AlexxIT/go2rtc>
- [15] MDN Web Docs, Media Source Extensions API. [Online]. Elérhető: https://developer.mozilla.org/en-US/docs/Web/API/Media_Source_Extensions_API
- [16] J. Redmon, S. Divvala, R. Girshick, A. Farhadi, „You Only Look Once: Unified, Real-Time Object Detection”, in Proc. IEEE Conf. on Computer Vision and Pattern Recognition (CVPR), Las Vegas, NV, 2016, pp. 779–788.
- [17] S. Ren, K. He, R. Girshick, J. Sun, „Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks”, in Advances in Neural Information Processing Systems 28 (NIPS), Montréal, 2015, pp. 91–99.
- [18] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C.-Y. Fu, A. C. Berg, „SSD: Single Shot MultiBox Detector”, in Proc. European Conf. on Computer Vision (ECCV), Amsterdam, 2016, pp. 21–37.

- [19] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, C. L. Zitnick, „Microsoft COCO: Common Objects in Context”, in Proc. European Conf. on Computer Vision (ECCV), Zürich, 2014, pp. 740–755.
- [20] Z. Zivkovic, „Improved Adaptive Gaussian Mixture Model for Background Subtraction”, in Proc. 17th Int. Conf. on Pattern Recognition (ICPR), Cambridge, 2004, vol. 2, pp. 28–31.
- [21] H. Schulzrinne, A. Rao, R. Lanphier, Real Time Streaming Protocol (RTSP), RFC 2326, IETF, 1998. [Online]. Elérhető: <https://www.rfc-editor.org/rfc/rfc2326>
- [22] H. Schulzrinne, S. Casner, R. Frederick, V. Jacobson, RTP: A Transport Protocol for Real-Time Applications, RFC 3550, IETF, 2003. [Online]. Elérhető: <https://www.rfc-editor.org/rfc/rfc3550>
- [23] M. Handley, V. Jacobson, C. Perkins, SDP: Session Description Protocol, RFC 4566, IETF, 2006. [Online]. Elérhető: <https://www.rfc-editor.org/rfc/rfc4566>
- [24] I. Fette, A. Melnikov, The WebSocket Protocol, RFC 6455, IETF, 2011. [Online]. Elérhető: <https://www.rfc-editor.org/rfc/rfc6455>
- [25] WHATWG, HTML Living Standard – Server-sent events. [Online]. Elérhető: <https://html.spec.whatwg.org/multipage/server-sent-events.html>
- [26] R. T. Fielding, Architectural Styles and the Design of Network-based Software Architectures. PhD értekezés, University of California, Irvine, 2000.
- [27] ITU-T Recommendation H.264, Advanced video coding for generic audiovisual services. International Telecommunication Union, 2003 (és későbbi kiadásai).
- [28] M. Jones, J. Bradley, N. Sakimura, JSON Web Token (JWT), RFC 7519, IETF, 2015. [Online]. Elérhető: <https://www.rfc-editor.org/rfc/rfc7519>
- [29] Android Developers, Camera2 API – Documentation. [Online]. Elérhető: <https://developer.android.com/reference/android/hardware/camera2/package-summary>
- [30] Eclipse Foundation, Eclipse Mosquitto – Documentation. [Online]. Elérhető: <https://mosquitto.org/documentation/>
- [31] NVIDIA, TensorRT – Documentation. [Online]. Elérhető: <https://docs.nvidia.com/deeplearning/tensorrt/>
- [32] Google, Coral Edge TPU – Documentation. [Online]. Elérhető: <https://coral.ai/docs/>
- [33] SQLite Consortium, SQLite – Documentation. [Online]. Elérhető: <https://www.sqlite.org/docs.html>
- [34] F5/NGINX, nginx – Documentation. [Online]. Elérhető: <https://nginx.org/en/docs/>

5. Mellékletek

A mellékletek a dolgozat fő fejezeteit feleslegesen terhelő kiegészítő anyagokat tartalmazzák.

5.1. Nyomtatott mellékletek

Jelen szakdolgozathoz különálló nyomtatott melléklet nem tartozik; az ábrák és táblázatok a dolgozat szövegében szerepelnek.

5.2. Elektronikus mellékletek

DVD, mely tartalmazza a teljes szakdolgozatot *.docx és *.pdf formátumban, valamint azt a prezentációt (*.pptx), melyet a hallgató a szakdolgozata védeke során használ. A lemezre a SwarmCam rendszer teljes forráskódja is felkerül; a forráskód nyilvános tárolóban is elérhető, a dolgozat leadásakor állapotában: <https://github.com/RiscyX/swarmcam>